

Dynamic Graph CNN for Learning on Point Clouds

YUE WANG, Massachusetts Institute of Technology

YONGBIN SUN, Massachusetts Institute of Technology

ZIWEI LIU, UC Berkeley / ICSI

SANJAY E. SARMA, Massachusetts Institute of Technology

MICHAEL M. BRONSTEIN, Imperial College London / USI Lugano

JUSTIN M. SOLOMON, Massachusetts Institute of Technology

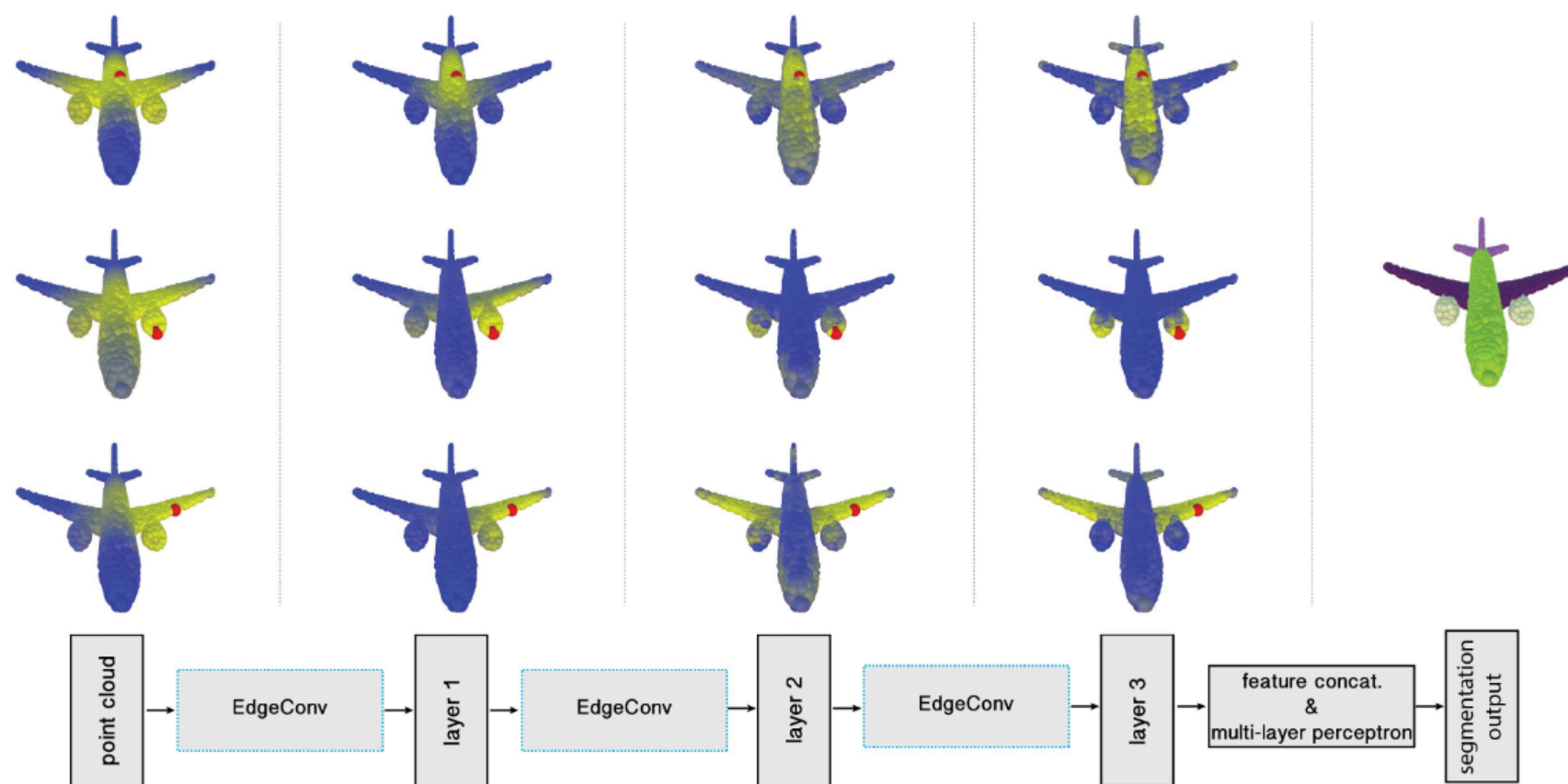


Fig. 1. **Point cloud segmentation using the proposed neural network.** Bottom: schematic neural network architecture. Top: Structure of the feature spaces produced at different layers of the network, visualized as the distance from the red point to all the rest of the points (shown left-to-right are the input and layers 1-3; rightmost figure shows the resulting segmentation). Observe how the feature space structure in deeper layers captures semantically similar structures such as wings, fuselage, or turbines, despite a large distance between them in the original input space.

Point clouds provide a flexible geometric representation suitable for countless applications in computer graphics; they also comprise the raw output of most 3D data acquisition devices. While hand-designed features on point clouds have long been proposed in graphics and vision, however, the recent overwhelming success of convolutional neural networks (CNNs) for image analysis suggests the value of adapting insight from CNN to the point cloud world. Point clouds inherently lack topological information so designing

Authors' addresses: Yue Wang, Massachusetts Institute of Technology, yuewang@csail.mit.edu; Yongbin Sun, Massachusetts Institute of Technology, yb_sun@mit.edu; Ziwei Liu, UC Berkeley / ICSI, zwliu@icsi.berkeley.edu; Sanjay E. Sarma, Massachusetts Institute of Technology, sesarma@mit.edu; Michael M. Bronstein, Imperial College London / USI Lugano, m.bronstein@imperial.ac.uk; Justin M. Solomon, Massachusetts Institute of Technology, jsolomon@mit.edu.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM. 0730-0301/2019/1-ART1 \$15.00
<https://doi.org/10.1145/3326362>

a model to recover topology can enrich the representation power of point clouds. To this end, we propose a new neural network module dubbed *EdgeConv* suitable for CNN-based high-level tasks on point clouds including classification and segmentation. *EdgeConv* acts on graphs dynamically computed in each layer of the network. It is differentiable and can be plugged into existing architectures. Compared to existing modules operating in extrinsic space or treating each point independently, *EdgeConv* has several appealing properties: It incorporates local neighborhood information; it can be stacked applied to learn global shape properties; and in multi-layer systems affinity in feature space captures semantic characteristics over potentially long distances in the original embedding. We show the performance of our model on standard benchmarks including ModelNet40, ShapeNetPart, and S3DIS.

CCS Concepts: • **Computing methodologies** → **Neural networks**; **Point-based models**; *Shape analysis*;

Additional Key Words and Phrases: point cloud, classification, segmentation

ACM Reference Format:

Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. 2019. Dynamic Graph CNN for Learning on Point Clouds. *ACM Trans. Graph.* 1, 1, Article 1 (January 2019), 13 pages. <https://doi.org/10.1145/3326362>

点云学习的动态图卷积神经网络

王越，麻省理工学院 孙勇斌，麻省理工学院 刘子伟，加州大学伯克利分校 / ICSI 桑杰·E·萨马，麻省理工学院 布鲁斯汀·M·布隆斯坦，伦敦帝国学院 / USI 卢加诺 詹姆斯·M·索洛蒙，麻省理工学院

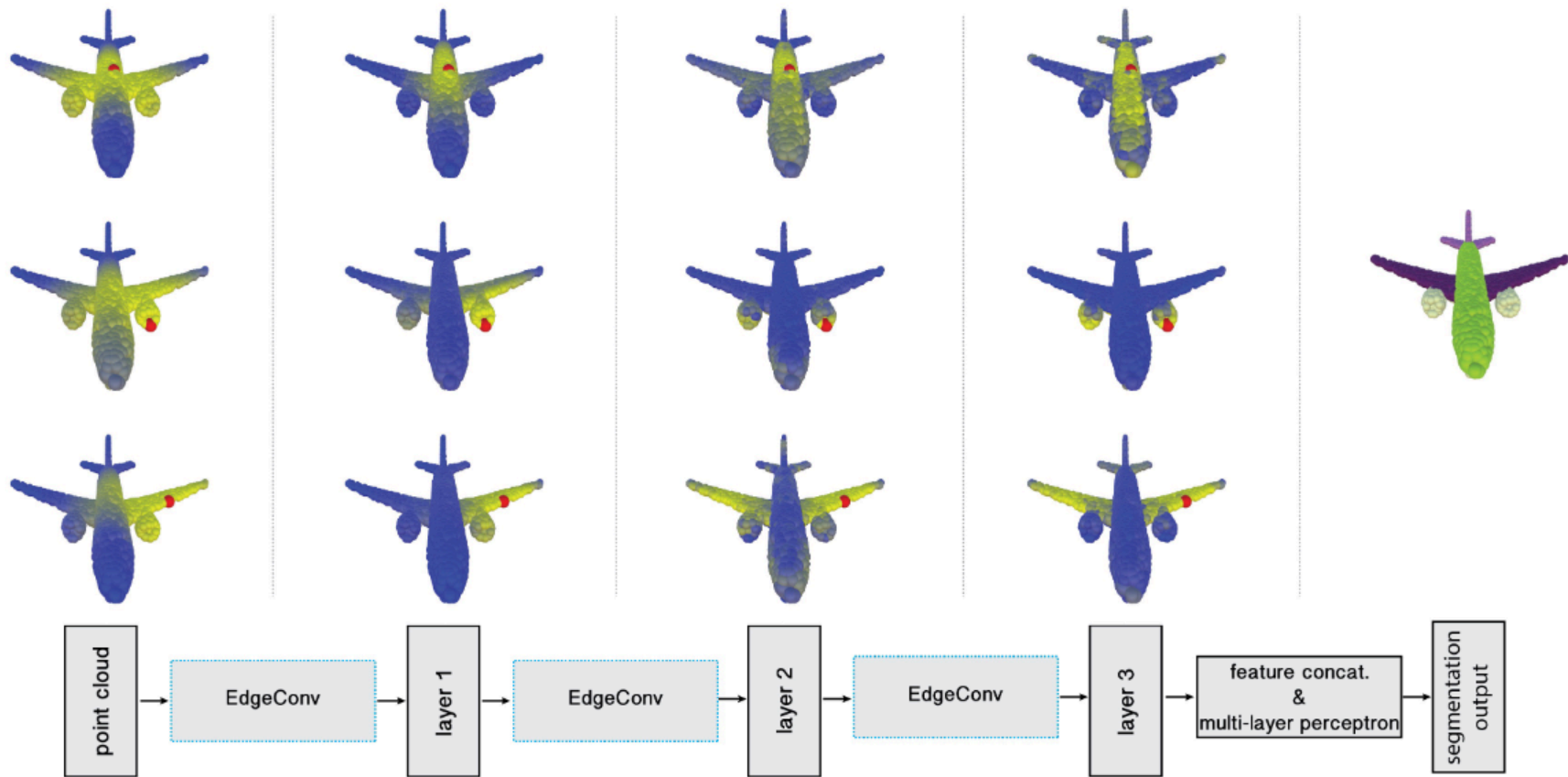


图 1. 使用所提出的神经网络进行点云分割。底部：神经网络架构示意图。顶部：网络不同层产生的特征空间结构，可视化为红点到所有其他点的距离（从左到右显示输入和第 1-3 层；最右边的图显示了结果分割）。观察深层特征空间结构如何捕获语义相似的形状，如机翼、机身或涡轮，尽管它们在原始输入空间中相距较远。

点云提供了一种灵活的几何表示形式，适用于计算机图形学中的无数应用；它们也是大多数三维数据采集设备的原始输出。虽然图形和视觉领域长期以来一直提出在点云上设计手工特征，然而，卷积神经网络（CNN）在图像分析方面的巨大成功表明，将 CNN 的见解应用于点云领域具有重要价值。点云本质上缺乏拓扑信息，因此设计

作者地址：Yue Wang, 麻省理工学院, yuewang@csail.mit.edu; Yongbin Sun, 麻省理工学院, yb_sun@mit.edu; Ziwei Liu, 加州大学伯克利分校 / ICSI, zwliu@icsi.berkeley.edu; Sanjay E. Sarma, 麻省理工学院, sesarma@mit.edu; Michael M. Bronstein, 伦敦帝国学院 / USI Lugano, m.bronstein@imperial.ac.uk; Justin M. Solomon, 麻省理工学院, jsolomon@mit.edu.

使用个人或课堂用途，复制本工作全部或部分内容，无需付费，但须保证复制品不用于盈利或商业优势，且复制品需包含本声明并在首页完整注明引用信息。本工作中由非作者所有部分的所有权必须得到尊重。经注明出处后，允许摘录。其他形式的复制或再出版，或发布在服务器上，或分发到列表中，需事先获得特定许可和/或付费。请向 permissions@acm.org 申请许可。

一个用于恢复拓扑结构的模型可以增强点云的表示能力。为此，我们提出了一种名为 EdgeConv 的新型神经网络模块，适用于点云上的基于 CNN 的高级任务，包括分类和分割。EdgeConv 作用于网络每一层动态计算的图。它是可微分的，可以嵌入到现有架构中。与在 extrinsic 空间中操作或独立处理每个点的现有模块相比，EdgeConv 具有几个吸引人的特性：它包含了局部邻域信息；它可以堆叠应用以学习全局形状属性；在多层系统中，特征空间中的亲和力可以捕捉原始嵌入中潜在在长距离的语义特征。我们在包括 ModelNet40、ShapeNetPart 和 S3DIS 的标准基准测试中展示了我们模型的性能。

CCS 概念：· 计算方法 → 神经网络；点
基于模型；形状分析；

其他关键词和短语：点云，分类，分割

ACM 参考格式：
Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E. Sarma, Michael M. Bronstein, and Justin M. Solomon. 2019. Dynamic Graph CNN for Learning on Point Clouds. ACM Trans. Graph. 1, 1, Article 1 (January 2019), 13 pages. <https://doi.org/10.1145/3326362>

1 INTRODUCTION

Point clouds, or scattered collections of points in 2D or 3D, are arguably the simplest shape representation; they also comprise the output of 3D sensing technology including LiDAR scanners and stereo reconstruction. With the advent of fast 3D point cloud acquisition, recent pipelines for graphics and vision often process point clouds directly, bypassing expensive mesh reconstruction or denoising due to efficiency considerations or instability of these techniques in the presence of noise. A few of the many recent applications of point cloud processing and analysis include indoor navigation [Zhu et al. 2017], self-driving vehicles [Liang et al. 2018; Qi et al. 2017a; Wang et al. 2018b], robotics [Rusu et al. 2008b], and shape synthesis and modeling [Golovinskiy et al. 2009; Guerrero et al. 2018].

These modern applications demand *high-level* processing of point clouds. Rather than identifying salient geometric features like corners and edges, recent algorithms search for semantic cues and affordances. These features do not fit cleanly into the frameworks of computational or differential geometry and typically require learning-based approaches that derive relevant information through statistical analysis of labeled or unlabeled datasets.

In this paper, we primarily consider point cloud classification and segmentation, two model tasks in point cloud processing. Traditional methods for solving these problems employ handcrafted features to capture geometric properties of point clouds [Lu et al. 2014; Rusu et al. 2009, 2008a]. More recently, the success of deep neural networks for image processing has motivated a data-driven approach to learning features on point clouds. Deep point cloud processing and analysis methods are developing rapidly and outperform traditional approaches in various tasks [Chang et al. 2015].

Adaptation of deep learning to point cloud data, however, is far from straightforward. Most critically, standard deep neural network models require input data with regular structure, while point clouds are fundamentally irregular: Point positions are continuously distributed in the space, and any permutation of their ordering does not change the spatial distribution. One common approach to process point cloud data using deep learning models is to first convert raw point cloud data into a volumetric representation, namely a 3D grid [Maturana and Scherer 2015; Wu et al. 2015]. This approach, however, usually introduces quantization artifacts and excessive memory usage, making it difficult to go to capture high-resolution or fine-grained features.

State-of-the-art deep neural networks are designed specifically to handle the irregularity of point clouds, directly manipulating raw point cloud data rather than passing to an intermediate regular representation. This approach was pioneered by *PointNet* [Qi et al. 2017b], which achieves permutation invariance of points by operating on each point independently and subsequently applying a symmetric function to accumulate features. Various extensions of *PointNet* consider neighborhoods of points rather than acting on each independently [Qi et al. 2017c; Shen et al. 2017]; these allow the network to exploit local features, improving upon performance of the basic model. These techniques largely treat points independently at local

scale to maintain permutation invariance. This independence, however, neglects the geometric relationships among points, presenting a fundamental limitation that cannot capture local features.

To address these drawbacks, we propose a novel simple operation, called *EdgeConv*, which captures local geometric structure while maintaining permutation invariance. Instead of generating point features directly from their embeddings, *EdgeConv* generates *edge features* that describe the relationships between a point and its neighbors. *EdgeConv* is designed to be invariant to the ordering of neighbors, and thus is permutation invariant. Because *EdgeConv* explicitly constructs a local graph and learns the embeddings for the edges, the model is capable of grouping points both in Euclidean space and in semantic space.

EdgeConv is easy to implement and integrate into existing deep learning models to improve their performance. In our experiments, we integrate *EdgeConv* into the basic version of *PointNet* without using any feature transformation. We show the resulting network achieves state-of-the-art performance on several datasets, most notably *ModelNet40* and *S3DIS* for classification and segmentation.

Key Contributions. We summarize the key contributions of our work as follows:

- We present a novel operation for learning from point clouds, *EdgeConv*, to better capture local geometric features of point clouds while still maintaining permutation invariance.
- We show the model can learn to semantically group points by dynamically updating a graph of relationships from layer to layer.
- We demonstrate that *EdgeConv* can be integrated into multiple existing pipelines for point cloud processing.
- We present extensive analysis and testing of *EdgeConv* and show that it achieves state-of-the-art performance on benchmark datasets.
- We release our code to facilitate reproducibility and future research.¹

2 RELATED WORK

Hand-Crafted Features. Various tasks in geometric data processing and analysis—including segmentation, classification, and matching—require some notion of local similarity between shapes. Traditionally, this similarity is established by constructing feature descriptors that capture local geometric structure. Countless papers in computer vision and graphics propose local feature descriptors for point clouds suitable for different problems and data structures. A comprehensive overview of hand-designed point features is out of the scope of this paper, but we refer the reader to [Biasotti et al. 2016; Guo et al. 2014; Van Kaick et al. 2011] for discussion.

Broadly speaking, one can distinguish between *extrinsic* and *intrinsic* descriptors. Extrinsic descriptors usually are derived from the coordinates of the shape in 3D space and includes classical methods like shape context [Belongie et al. 2001], spin images [Johnson and Hebert 1999], integral features [Manay et al. 2006], distance-based descriptors [Ling and Jacobs 2007], point feature histograms [Rusu et al. 2009, 2008a], and normal histograms [Tombari et al. 2011], to name a few. Intrinsic descriptors treat the 3D shape as a manifold whose metric structure is discretized as a mesh or graph; quantities

¹<https://github.com/WangYueFt/dgcnn>

1 引言

点云，即二维或三维空间中的散乱点集，可以说是最简单的形状表示方法；它们也是包括激光雷达扫描器和立体重建在内的三维传感技术的输出。随着快速三维点云采集技术的出现，近期的图形和视觉处理流程通常直接处理点云，绕过昂贵的网格重建或去噪，这是出于效率考虑或这些技术在噪声存在时的不稳定性。点云处理和分析的许多近期应用包括室内导航[Zhu 等人，2017 年]、自动驾驶汽车[Liang 等人，2018 年；Qi 等人，2017a；Wang 等人，2018b]、机器人[Rusu 等人，2008b]以及形状合成和建模[Golovinskiy 等人，2009 年；Guerrero 等人，2018 年]。

这些现代应用需要高层次的点云处理。与识别角点、边缘等显著几何特征不同，近期算法在寻找语义线索和可供性。这些特征无法完全契合计算几何或微分几何的框架，通常需要基于学习的处理方法，通过分析标记或未标记数据集的统计信息来提取相关信息。

在本文中，我们主要考虑点云分类和分割，这是点云处理中的两种模型任务。解决这些问题的传统方法采用手工特征来捕捉点云的几何属性[Lu 等人，2014；Rusu 等人，2009，2008a]。最近，图像处理中深度神经网络的成功促使人们采用数据驱动的方法在点云上学习特征。深度点云处理和分析方法正在迅速发展，并在各种任务中超越了传统方法[Chang 等人，2015]。

将深度学习应用于点云数据，然而远非易事。最关键的是，标准的深度神经网络模型需要输入具有规则结构的数据，而点云本质上是不规则的：点位置在空间中连续分布，并且它们顺序的任何排列都不会改变空间分布。使用深度学习模型处理点云数据的一种常见方法是首先将原始点云数据转换为体素表示，即 3D 网格[Maturana and Scherer 2015; Wu et al. 2015]。然而，这种方法通常会引入量化伪影和过度的内存使用，使得难以捕捉高分辨率或细粒度特征。

最先进的深度神经网络专门设计用于处理点云的不规则性，直接操作原始点云数据而不是将其传递到一个中间的规则表示。这种方法由 PointNet [Qi 等人 2017b]首创，它通过独立地对每个点进行操作，然后应用对称函数来累积特征，从而实现了点的置换不变性。PointNet 的各种扩展考虑了点的邻域，而不是独立地对每个点进行操作[Qi 等人 2017c；Shen 等人 2017]；这些扩展允许网络利用局部特征，从而改进了基本模型的性能。

这些技术主要在局部尺度上独立地处理点以保持置换不变性。然而，这种独立性忽略了点之间的几何关系，存在一个根本性的限制，即无法捕捉局部特征。

为解决这些缺点，我们提出了一种新颖的简单操作，称为 EdgeConv，它能够捕获局部几何结构同时保持置换不变性。EdgeConv 不是直接从其嵌入生成点特征，而是生成边特征来描述一个点与其邻居之间的关系。EdgeConv 设计为对邻居的顺序不变，因此具有置换不变性。由于 EdgeConv 明确构建局部图并学习边的嵌入，该模型能够对欧几里得空间和语义空间中的点进行分组。

EdgeConv 易于实现并集成到现有的深度学习模型中以提升其性能。在我们的实验中，我们将 EdgeConv 集成到未经任何特征转换的 PointNet 基本版本中。我们展示了该网络在多个数据集上实现了最先进的性能，尤其是在分类和分割任务中，ModelNet40 和 S3DIS 表现尤为突出。

主要贡献。我们总结我们工作的主要贡献如下：

- 我们提出了一种用于从点云中学习的新型操作 EdgeConv，以更好地捕获点云的局部几何特征，同时仍然保持置换不变性。
- 我们展示了该模型能够通过逐层动态更新关系图来学习语义分组点。
- 我们证明了 EdgeConv 可以集成到多个现有的点云处理流程中。
- 我们展示了 EdgeConv 的广泛分析和测试，并表明它在基准数据集上达到了最先进的性能。
- 我们发布我们的代码以促进可复现性和未来研究。

2 相关工作 手工特征。几何数据处理和分析中的各种任务——包括分割、分类和匹配——需要形状之间某种局部相似性的概念。传统上，这种相似性是通过构建捕获局部几何结构的特征描述符来建立的。计算机视觉和图形领域的无数论文提出了适用于不同问题和数据结构的点云局部特征描述符。手工设计的点特征的全面概述超出了本文的范围，但我们建议读者参考[Biasotti 等人，2016；Guo 等人，2014；Van Kaick 等人，2011]进行讨论。

广义而言，可以将描述符分为外生和内生两种。外生描述符通常由三维空间中形状的坐标导出，包括形状上下文[Belongie 等人.2001]、旋转图像[Johnson 和 Hebert 1999]、积分特征[Manay 等人.2006]、基于距离的描述符[Ling 和 Jacobs 2007]、点特征直方图[Rusu 等人 . 2009, 2008a]和法线直方图[Tombari 等人 . 2011]等经典方法。内生描述符将三维形状视为一个流形，其度量结构被离散化为网格或图；量

1<https://github.com/WangYueFt/dgcnn>

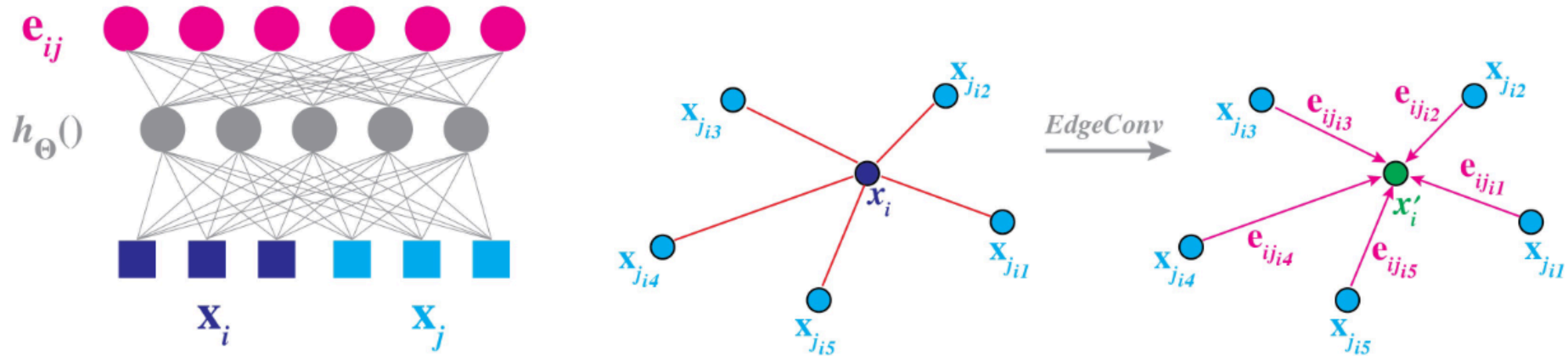


Fig. 2. **Left:** Computing an edge feature, e_{ij} (top), from a point pair, x_i and x_j (bottom). In this example, $h_{\Theta}()$ is instantiated using a fully connected layer, and the learnable parameters are its associated weights. **Right:** The EdgeConv operation. The output of EdgeConv is calculated by aggregating the edge features associated with all the edges emanating from each connected vertex.

expressed in terms of the metric are invariant to isometric deformation. Representatives of this class include spectral descriptors such as global point signatures [Rustamov 2007], the heat and wave kernel signatures [Aubry et al. 2011; Sun et al. 2009], and variants [Bronstein and Kokkinos 2010]. Most recently, several approaches wrap machine learning schemes around standard descriptors [Guo et al. 2014; Shah et al. 2013].

Deep learning on geometry. Following the breakthrough results of convolutional neural networks (CNNs) in vision [Krizhevsky et al. 2012; LeCun et al. 1989], there has been strong interest to adapt such methods to geometric data. Unlike images, geometry usually does not have an underlying grid, requiring new building blocks replacing convolution and pooling or adaptation to a grid structure.

As a simple way to overcome this issue, view-based [Su et al. 2015; Wei et al. 2016] and volumetric representations [Klokov and Lempitsky 2017; Maturana and Scherer 2015; Tatarchenko et al. 2017; Wu et al. 2015]—or their combination [Qi et al. 2016]—“place” geometric data onto a grid. More recently, PointNet [Qi et al. 2017b,c] exemplifies a broad class of deep learning architectures on non-Euclidean data (graphs and manifolds) termed *geometric deep learning* [Bronstein et al. 2017]. These date back to early methods to construct neural networks on graphs [Scarselli et al. 2009], recently improved with gated recurrent units [Li et al. 2016] and neural message passing [Gilmer et al. 2017]. Bruna et al. [2013] and Henaff et al. [2015] generalized convolution to graphs via the Laplacian eigenvectors [Shuman et al. 2013]. Computational drawbacks of this foundational approach were alleviated in follow-up works using polynomial [Deferrard et al. 2016; Kipf and Welling 2017; Monti et al. 2017b, 2018], or rational [Levie et al. 2017] spectral filters that avoid Laplacian eigendecomposition and guarantee localization. An alternative definition of non-Euclidean convolution employs spatial rather than spectral filters. The *Geodesic CNN (GCNN)* is a deep CNN on meshes generalizing the notion of patches using local intrinsic parameterization [Masci et al. 2015]. Its key advantage over spectral approaches is better generalization as well as a simple way of constructing directional filters. Follow-up work proposed different local charting techniques using anisotropic diffusion [Boscaini et al. 2016] or Gaussian mixture models [Monti et al. 2017a; Veličković et al. 2017]. In [Halimi et al. 2018; Litany et al. 2017b], a differentiable functional map [Ovsjanikov et al. 2012] layer was incorporated into

a geometric deep neural network, allowing to do intrinsic structured prediction of correspondence between nonrigid shapes.

The last class of geometric deep learning approaches attempts to pull back a convolution operation by embedding the shape into a domain with shift-invariant structure such as the sphere [Sinha et al. 2016], torus [Maron et al. 2017], plane [Ezuz et al. 2017], sparse network lattice [Su et al. 2018], or spline [Fey et al. 2018].

Finally, we should mention *geometric generative models*, which attempt to generalize models such as autoencoders, variational autoencoders (VAE) [Kingma and Welling 2013], and generative adversarial networks (GAN) [Goodfellow et al. 2014] to the non-Euclidean setting. One of the fundamental differences between these two settings is the lack of canonical order between the input and the output vertices, thus requiring an input-output correspondence problem to be solved. In 3D mesh generation, it is commonly assumed that the mesh is given and its vertices are canonically ordered; the generation problem thus amounts only to determining the embedding of the mesh vertices. Kostrikov et al. [2017] proposed SurfaceNets based on the extrinsic Dirac operator for this task. Litany et al. [2017a] introduced the intrinsic VAE for meshes and applied it to shape completion; a similar architecture was used by Ranjan et al. [2018] for 3D face synthesis. For point clouds, multiple generative architectures have been proposed [Fan et al. 2017; Li et al. 2018b; Yang et al. 2018].

3 OUR APPROACH

We propose an approach inspired by PointNet and convolution operations. Instead of working on individual points like PointNet, however, we exploit local geometric structures by constructing a local neighborhood graph and applying convolution-like operations on the edges connecting neighboring pairs of points, in the spirit of graph neural networks. We show in the following that such an operation, dubbed *edge convolution* (EdgeConv), has properties lying between translation-invariance and non-locality.

Unlike graph CNNs, our graph is not fixed but rather is dynamically updated after each layer of the network. That is, the set of k -nearest neighbors of a point changes from layer to layer of the network and is computed from the sequence of embeddings. Proximity in feature space differs from proximity in the input, leading to nonlocal diffusion of information throughout the point cloud. As

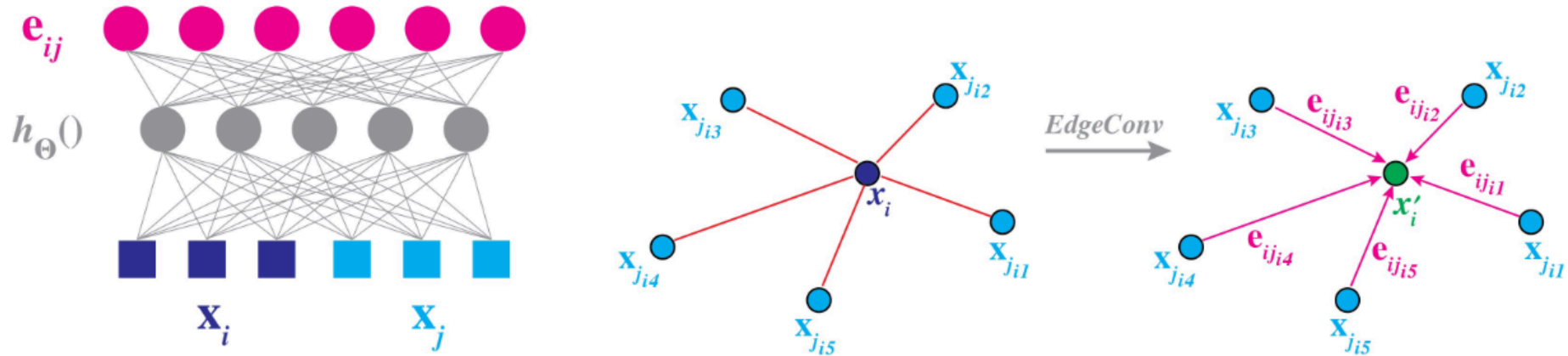


图 2。左：从一个点对 x 和 x (bottom) 计算边特征 e (top)。在这个例子中， $h(\cdot)$ 使用全连接层实例化，可学习参数是其相关的权重。右：EdgeConv 操作。EdgeConv 的输出是通过聚合从每个连接顶点发出的所有边关联的边特征来计算的。

在度量中表达的是对等距变形不变的。这类代表包括频谱描述符，如全局点签名[Rustamov 2007]、热和波核签名[Aubry 等人 2011; Sun 等人 2009]以及变体[Bronstein 和 Kokkinos 2010]。最近，几种方法将机器学习方案包装在标准描述符周围[Guo 等人 2014; Shah 等人 2013]。

几何上的深度学习。在卷积神经网络(CNNs)在视觉领域的突破性结果之后[Krizhevsky 等人 2012; LeCun 等人 1989]，人们很感兴趣将这些方法应用于几何数据。与图像不同，几何通常没有底层网格，需要新的构建块来替代卷积和池化，或者适应网格结构。

作为解决这个问题的一种简单方法，基于视图的表示 [Su 等人, 2015; Wei 等人, 2016] 和体素表示 [Klokov 和 Lempitsky, 2017; Maturana 和 Scherer, 2015; Tatarchenko 等人, 2017; Wu 等人, 2015]——或它们的组合 [Qi 等人, 2016]——将几何数据放置到网格上。最近，PointNet [Qi 等人, 2017b,c] 作为一个广泛类别的非欧几里得数据（图和流形）的深度学习架构的例子，被称为几何深度学习 [Bronstein 等人, 2017]。这些可以追溯到早期在图上构建神经网络的方法 [Scarselli 等人, 2009]，最近通过门控循环单元 [Li 等人, 2016] 和神经消息传递 [Gilmer 等人, 2017] 得到改进。Bruna 等人 [2013] 和 Henaff 等人 [2015] 通过拉普拉斯特征向量 [Shuman 等人, 2013] 将卷积推广到图上。这个基础方法的计算缺点在后续工作中通过多项式 [Defferrard 等人, 2016; Kipf 和 Welling, 2017; Monti 等人, 2017b, 2018] 或有理 [Levie 等人, 2017] 光谱滤波器得到缓解，这些滤波器避免了拉普拉斯特征分解并保证了定位。一种非欧几里得卷积的替代定义使用空间滤波器而非频谱滤波器。Geodesic CNN

(GCNN) 是一种在网格上的深度卷积神经网络，它通过局部内在参数化推广了块的概念[Masci 等人 2015]。其相对于频谱方法的关键优势在于更好的泛化能力以及构建方向滤波器的简单方法。后续工作提出了使用各向异性扩散[Boscaini 等人 2016]或高斯混合模型[Monti 等人 2017a; Velickovic等人 2017]的不同局部制图技术。在[Halimi 等人 2018; Litany 等人 2017b]中，提出了一种可微分的函数映射 [Ovsjanikov 等人

2012]层被整合进几何深度神经网络，从而能够对非刚性形状之间的对应关系进行内在结构预测。

最后一类几何深度学习方法试图通过将形状嵌入具有平移不变结构的域中来拉回卷积操作，例如球体 [Sinha 等人, 2016 年]、环面 [Maron 等人, 2017 年]、平面 [Ezuz 等人, 2017 年]、稀疏网络晶格 [Su 等人, 2018 年] 或样条 [Fey 等人, 2018 年]。

最后，我们应提及几何生成模型，这些模型试图将自编码器、变分自编码器 (VAE) [Kingma and Welling 2013] 和生成对抗网络 (GAN) [Goodfellow et al. 2014] 等模型推广到非欧几里得设置。这两种设置之间的一个基本区别是输入和输出顶点之间缺乏规范顺序，因此需要解决输入输出对应问题。在 3D 网格生成中，通常假设网格是给定的，并且其顶点是规范排序的；因此，生成问题仅归结为确定网格顶点的嵌入。Kostrikov 等人 [2017] 基于外尔狄拉克算子提出了用于此任务的 SurfaceNets。Litany 等人 [2017a] 引入了用于网格的内在 VAE，并将其应用于形状补全；Ranjan 等人 [2018] 使用了类似的架构进行 3D 人脸合成。对于点云，已经提出了多种生成架构 [Fan et al. 2017; Li et al. 2018b; Yang et al. 2018]。

3 我们的方案

我们提出了一种受 PointNet 和卷积操作启发的方案。然而，与 PointNet 在单个点上工作不同，我们通过构建局部邻域图来利用局部几何结构，并在连接相邻点对的边上进行类似卷积的操作，这体现了图神经网络的精神。我们在下文将证明，这种被称为边卷积 (EdgeConv) 的操作，具有介于平移不变性和非局部性之间的特性。

与图卷积网络不同，我们的图不是固定的，而是在网络的每一层之后动态更新。也就是说，一个点的 k 个最近邻在网络的每一层中都会发生变化，并且是从嵌入序列中计算的。特征空间中的邻近性与输入中的邻近性不同，导致信息在整个点云中非局部地扩散。

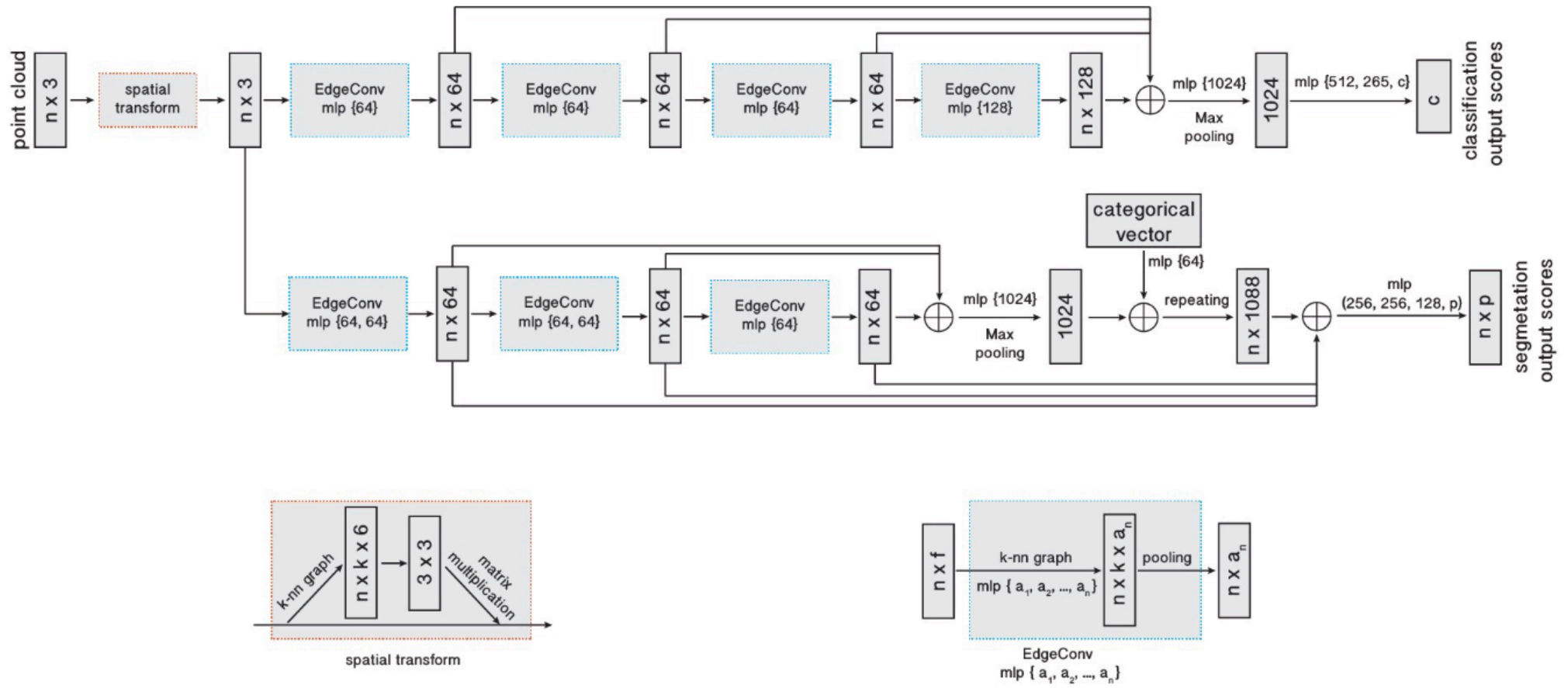


Fig. 3. **Model architectures:** The model architectures used for classification (top branch) and segmentation (bottom branch). The classification model takes as input n points, calculates an edge feature set of size k for each point at an EdgeConv layer, and aggregates features within each set to compute EdgeConv responses for corresponding points. The output features of the last EdgeConv layer are aggregated globally to form an 1D global descriptor, which is used to generate classification scores for c classes. The segmentation model extends the classification model by concatenating the 1D global descriptor and all the EdgeConv outputs (serving as local descriptors) for each point. It outputs per-point classification scores for p semantic labels. $\oplus$: concatenation. **Point cloud transform block:** The point cloud transform block is designed to align an input point set to a canonical space by applying an estimated 3×3 matrix. To estimate the 3×3 matrix, a tensor concatenating the coordinates of each point and the coordinate differences between its k neighboring points is used. **EdgeConv block:** The EdgeConv block takes as input a tensor of shape $n \times f$, computes edge features for each point by applying a multi-layer perceptron (mlp) with the number of layer neurons defined as $\{a_1, a_2, \dots, a_n\}$, and generates a tensor of shape $n \times a_n$ after pooling among neighboring edge features.

a connection to existing work, Non-local Neural Networks [Wang et al. 2018a] explored similar ideas in the video recognition field, and follow-up work by Xie et al. [2018] proposed using non-local blocks to denoise feature maps to defend against adversarial attacks.

3.1 Edge Convolution

Consider an F -dimensional point cloud with n points, denoted by $\mathbf{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_n\} \subseteq \mathbb{R}^F$. In the simplest setting of $F = 3$, each point contains 3D coordinates $\mathbf{x}_i = (x_i, y_i, z_i)$; it is also possible to include additional coordinates representing color, surface normal, and so on. In a deep neural network architecture, each subsequent layer operates on the output of the previous layer, so more generally the dimension F represents the feature dimensionality of a given layer.

We compute a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ representing local point cloud structure, where $\mathcal{V} = \{1, \dots, n\}$ and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ are the *vertices* and *edges*, respectively. In the simplest case, we construct $\mathcal{G}$ as the k -nearest neighbor (k -NN) graph of $\mathbf{X}$ in $\mathbb{R}^F$. The graph includes self-loop, meaning each node also points to itself. We define *edge features* as $\mathbf{e}_{ij} = h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j)$, where $h_{\Theta} : \mathbb{R}^F \times \mathbb{R}^F \rightarrow \mathbb{R}^{F'}$ is a nonlinear function with a set of learnable parameters Θ .

Finally, we define the EdgeConv operation by applying a channel-wise symmetric aggregation operation $\square$ (e.g., $\sum$ or $\max$) on the edge features associated with all the edges emanating from each

vertex. The output of EdgeConv at the i -th vertex is thus given by

$$\mathbf{x}'_i = \square_{j:(i,j) \in \mathcal{E}} h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j). \quad (1)$$

Making analogy to convolution along images, we regard $\mathbf{x}_i$ as the central pixel and $\{\mathbf{x}_j : (i, j) \in \mathcal{E}\}$ as a patch around it (see Figure 2). Overall, given an F -dimensional point cloud with n points, EdgeConv produces an F' -dimensional point cloud with the same number of points.

Choice of h and $\square$. The choice of the edge function and the aggregation operation has a crucial influence on the properties of EdgeConv. For example, when $\mathbf{x}_1, \dots, \mathbf{x}_n$ represent image pixels on a regular grid and the graph $\mathcal{G}$ has connectivity representing patches of fixed size around each pixel, the choice $\theta_m \cdot \mathbf{x}_j$ as the edge function and sum as the aggregation operation yields standard convolution:

$$\mathbf{x}'_{im} = \sum_{j:(i,j) \in \mathcal{E}} \theta_m \cdot \mathbf{x}_j, \quad (2)$$

Here, $\Theta = (\theta_1, \dots, \theta_M)$ encodes the weights of M different filters. Each θ_m has the same dimensionality as $\mathbf{x}$, and $\cdot$ denotes the Euclidean inner product.

A second choice of h is

$$h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = h_{\Theta}(\mathbf{x}_i), \quad (3)$$

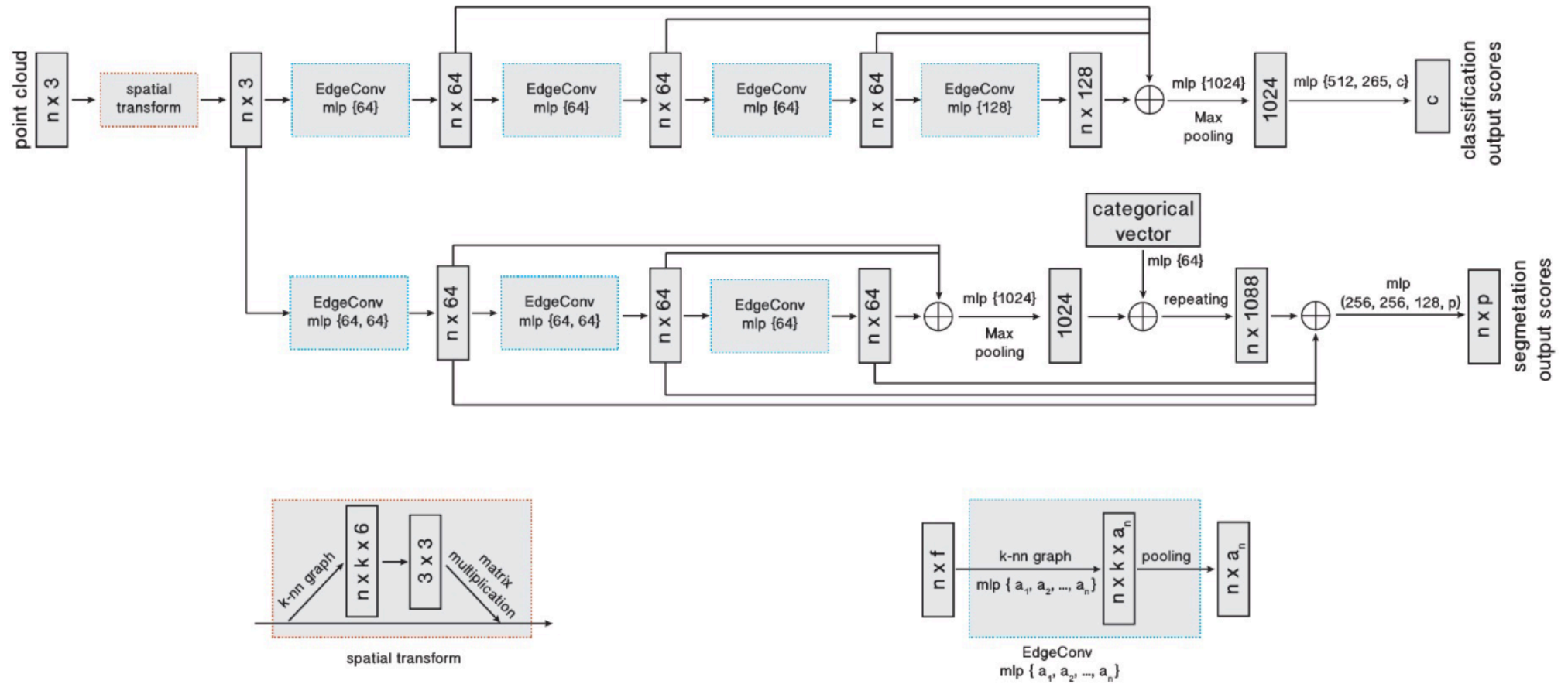


图 3. 模型架构：用于分类（上分支）和分割（下分支）的模型架构。分类模型以 n 个点为输入，在每个 EdgeConv 层为每个点计算大小为 k 的边特征集，并将每个集合内的特征聚合以计算对应点的 EdgeConv 响应。最后 EdgeConv 层的输出特征全局聚合形成 1D 全局描述符，用于生成 c 个类别的分类分数。分割模型通过将 1D 全局描述符和每个点的所有 EdgeConv 输出（作为局部描述符）连接来扩展分类模型，输出每个点的 p 个语义标签的分类分数。 $\oplus$ ：连接。点云变换模块：点云变换模块设计为通过应用估计的 3×3 矩阵将输入点集对齐到规范空间。为估计 3×3 矩阵，使用一个张量，该张量连接每个点的坐标及其 k 个邻近点之间的坐标差。EdgeConv 模块：EdgeConv 模块以形状为 $n \times f$ 的张量作为输入，通过应用具有层数神经元数量定义为 $\{a, a, \dots, a\}$ 的多层感知器（mlp）来计算每个点的边特征，并在相邻边特征之间进行池化后生成形状为 $n \times a$ 的张量。

与现有工作的联系，非局部神经网络[Wang 等人 2018a]在视频识别领域探索了类似的思想，Xie 等人[2018]的后续工作提出了使用非局部块来降噪特征图以防御对抗性攻击。

3.1 边缘卷积

考虑一个 F 维点云，包含 n 个点，记为 $X = \{x, \dots, x\} \subseteq \mathbb{R}^F$ 。在 $F=3$ 的最简单设置中，每个点包含 3D 坐标 $x=(x, y, z)$ ；也可以包含表示颜色、表面法线等的附加坐标。在深度神经网络架构中，每一层都基于前一层的输出进行操作，因此更一般地，维度 F 表示给定层的特征维度。我们计算一个有向图 $G = (V, E)$ 来表示局部点云结构，其中 $V = \{1, \dots, n\}$ 且 $E \subseteq V \times V$ 分别是顶点和边。在 simplest case 中，我们将 G 构建为 X 在 $\mathbb{R}$ 中的 k 近邻 (k -NN) 图。该图包含自环，即每个节点也指向自身。我们定义边特征为 $e = h(x, x)$ ，其中 $h: \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$

是一个具有一组可学习参数 Θ 的非线性函数。最后，我们通过应用一个通道对称的聚合操作 $\square$ （例如）来定义 EdgeConv 操作

在每条边关联的所有从每个顶点发出的边上的边特征上取 ($\bar{\cdot}$ 或 $\max$)。因此，EdgeConv 在第 i 个顶点的输出由

$$x = \square_{j:(i,j) \in E} h(x, x). \quad (1)$$

与图像卷积类比，我们将 x 视为中心像素，将 $\{x: (i, j) \in E\}$ 视为其周围的区域（见图 2）。总体而言，给定一个具有 n 个点的 F 维点云，EdgeConv 会生成一个具有相同点数的 F 维点云。

h 和 $\square$ 的选择。边函数和聚合操作的选择对 EdgeConv 的性质有重要影响。例如，当 $x, \dots, x$ 表示规则网格上的图像像素，且图 G 的连通性表示每个像素周围固定大小的区域时，将 $\theta \cdot x$ 作为边函数，将求和作为聚合操作，即可得到标准卷积：

$$x = \sum_{j:(i,j) \in E} \tilde{\theta} \cdot x, \quad (2)$$

这里， $\Theta = (\theta, \dots, \theta)$ 编码了 M 个不同滤波器的权重。每个 θ 的维度与 x 相同， $\cdot$ 表示欧几里得内积。

h 的第二种选择是

$$h(x, x) = h(x), \quad (3)$$

encoding only global shape information oblivious of the local neighborhood structure. This type of operation is used in PointNet, which can thus be regarded as a special case of EdgeConv.

A third choice of h adopted by Atzmon et al. [2018] is

$$h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = h_{\Theta}(\mathbf{x}_j) \quad (4)$$

and

$$x'_{im} = \sum_{j \in \mathcal{V}} (h_{\Theta}(\mathbf{x}_j)) g(u(\mathbf{x}_i, \mathbf{x}_j)), \quad (5)$$

where g is a Gaussian kernel and u computes pairwise distance in Euclidean space.

A fourth option is

$$h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = h_{\Theta}(\mathbf{x}_j - \mathbf{x}_i). \quad (6)$$

This encodes only local information, treating the shape as a collection of small patches and losing global structure.

Finally, a fifth option that we adopt in this paper is an asymmetric edge function

$$h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = \bar{h}_{\Theta}(\mathbf{x}_i, \mathbf{x}_j - \mathbf{x}_i). \quad (7)$$

This explicitly combines global shape structure, captured by the coordinates of the patch centers $\mathbf{x}_i$, with local neighborhood information, captured by $\mathbf{x}_j - \mathbf{x}_i$. In particular, we can define our operator by notating

$$e'_{ijm} = \text{ReLU}(\theta_m \cdot (\mathbf{x}_j - \mathbf{x}_i) + \phi_m \cdot \mathbf{x}_i), \quad (8)$$

which can be implemented as a shared MLP, and taking

$$x'_{im} = \max_{j: (i,j) \in \mathcal{E}} e'_{ijm}, \quad (9)$$

where $\Theta = (\theta_1, \dots, \theta_M, \phi_1, \dots, \phi_M)$

3.2 Dynamic graph update

Our experiments suggest that it is beneficial to *recompute the graph* using nearest neighbors in the feature space produced by each layer. This is a crucial distinction of our method from graph CNNs working on a fixed input graph. Such a dynamic graph update is the reason for the name of our architecture, the *Dynamic Graph CNN (DGCNN)*. With dynamic graph updates, the receptive field is as large as the diameter of the point cloud, while being sparse.

At each layer we have a different graph $\mathcal{G}^{(l)} = (\mathcal{V}^{(l)}, \mathcal{E}^{(l)})$, where the l -th layer edges are of the form $(i, j_{i1}), \dots, (i, j_{ik_l})$ such that $\mathbf{x}_{j_{i1}}^{(l)}, \dots, \mathbf{x}_{j_{ik_l}}^{(l)}$ are the k_l points closest to $\mathbf{x}_i^{(l)}$. Put differently, our architecture learns *how* to construct the graph $\mathcal{G}$ used in each layer rather than taking it as a fixed constant constructed before the network is evaluated. In our implementation, we compute a pairwise distance matrix in feature space and then take the closest k points for each single point.

3.3 Properties

Permutation Invariance. Consider the output of a layer

$$x'_i = \max_{j: (i,j) \in \mathcal{E}} h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) \quad (10)$$

and a permutation operator π . The output of the layer x'_i is invariant to permutation of the input $\mathbf{x}_j$ because max is a symmetric function (other symmetric functions also apply). The global max pooling operator to aggregate point features is also permutation-invariant.

Translation Invariance. Our operator has a “partial” translation invariance property, in that our choice of edge functions (7) explicitly exposes the part of the function that can be translation-dependent and optionally can be disabled. Consider a translation applied to $\mathbf{x}_j$ and $\mathbf{x}_i$; we can show that part of the edge feature is preserved when shifting by T . In particular, for the translated point cloud we have

$$\begin{aligned} e'_{ijm} &= \theta_m \cdot (\mathbf{x}_j + T - (\mathbf{x}_i + T)) + \phi_m \cdot (\mathbf{x}_i + T) \\ &= \theta_m \cdot (\mathbf{x}_j - \mathbf{x}_i) + \phi_m \cdot (\mathbf{x}_i + T). \end{aligned}$$

If we only consider $\mathbf{x}_j - \mathbf{x}_i$ by taking $\phi_m = 0$, then the operator is fully invariant to translation. In this case, however, the model reduces to recognizing an object based on an unordered set of patches, ignoring the positions and orientations of patches. With both $\mathbf{x}_j - \mathbf{x}_i$ and $\mathbf{x}_i$ as input, the model takes account into the local geometry of patches while keeping global shape information.

3.4 Comparison to existing methods

DGCNN is related to two classes of approaches, PointNet and graph CNNs, which we show to be particular settings of our method. We summarize different methods in Table 1.

PointNet is a special case of our method with $k = 1$, yielding a graph with an empty edge set $\mathcal{E} = \emptyset$. The edge function used in PointNet is $h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = h_{\Theta}(\mathbf{x}_i)$, which considers global but not local geometry. PointNet++ tries to account for local structure by applying PointNet in a local manner. In our parlance, PointNet++ first constructs the graph according to the Euclidean distances between the points, and in each layer applies a graph coarsening operation. For each layer, some points are selected using farthest point sampling (FPS); only the selected points are preserved while others are directly discarded after this layer. In this way, the graph becomes smaller after the operation applied on each layer. In contrast to DGCNN, PointNet++ computes pairwise distances using point input coordinates, and hence their graphs are fixed during training. The edge function used by PointNet++ is $h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = h_{\Theta}(\mathbf{x}_j)$, and the aggregation operation is also a max.

Among graph CNNs, MoNet [Monti et al. 2017a], ECC [Simonovsky and Komodakis 2017], Graph Attention Networks [Veličković et al. 2017], and the concurrent work [Atzmon et al. 2018] are the most related approaches. Their common denominator is a notion of a local patch on a graph, in which a convolution-type operation can be defined.²

Specifically, Monti et al. [2017a] use the graph structure to compute a local “pseudo-coordinate system” $\mathbf{u}$ in which the neighborhood vertices are represented; the convolution is then defined as an M -component Gaussian mixture

$$x'_{im} = \sum_{j: (i,j) \in \mathcal{E}} \theta_m \cdot (\mathbf{x}_j \odot g_{\mathbf{w}_n}(u(\mathbf{x}_i, \mathbf{x}_j))), \quad (11)$$

where g is a Gaussian kernel, $\odot$ is the elementwise (Hadamard) product, $\{\mathbf{w}_1, \dots, \mathbf{w}_N\}$ encode the learnable parameters of the Gaussians (mean and covariance), and $\{\theta_1, \dots, \theta_M\}$ are the learnable filter coefficients. (11) is an instance of our general operation (1), with a

²[Simonovsky and Komodakis 2017; Veličković et al. 2017] can be considered instances of [Monti et al. 2017a], with the difference that the weights are constructed employing features from adjacent nodes instead of graph structure; [Atzmon et al. 2018] is also similar except that the weighting function is hand-designed.

仅编码全局形状信息而忽略局部邻域结构。这种操作用于 PointNet，因此 PointNet 可以被视为 EdgeConv 的一种特例。

Atzmon 等人[2018]采用的 h 的第三种选择是

$$h(x, x) = h(x) \quad (4)$$

和

$$x = \bigoplus_{j \in V} (h) \varphi(u(x, x)), \quad (5)$$

其中 φ 是一个高斯核， u 计算欧几里得空间中的成对距离。

第四种选择是

$$h(x, x) = h(x - x). \quad (6)$$

这仅编码局部信息，将形状视为由小补丁组成的集合，并丢失全局结构。

最后，本文采用的一种第五种选项是对称边函数

$$h(x, x) = \frac{1}{2} h(x, x - x). \quad (7)$$

这明确地结合了全局形状结构，由补丁中心的坐标 x 捕获，以及局部邻域信息，由 $x - x$ 捕获。特别是，我们可以通过符号表示来定义我们的算子

$$e = \text{ReLU}(\theta \cdot (x - x) + \phi \cdot x), \quad (8)$$

这可以作为一个共享的 MLP 来实现，并且取

$$x = \max_{j: (i, j) \in E} e, \quad (9)$$

其中 $\Theta = (\theta, \dots, \theta, \phi, \dots, \phi)$

3.2 动态图更新

我们的实验表明，使用每一层产生的特征空间中的最近邻来重新计算图是有益的。这是我们方法与在固定输入图上工作的图卷积网络的关键区别。这种动态图更新是我们架构名称——动态图卷积网络（DGCNN）的原因。通过动态图更新，感受野的大小等于点云的直径，同时保持稀疏性。

在每一层，我们都有一个不同的图 $G = (V, E)$ ，其中第 l 层的边形式为 $(i, j), \dots, (i, j)$ ，使得 $x, \dots, x$

是离 x 最近的 k 个点。换句话说，我们的架构学习如何构建每层使用的图 G ，而不是将其作为网络评估前构建的固定常数。在我们的实现中，我们在特征空间中计算成对距离矩阵，然后为每个单个点取最近的 k 个点。

3.3 属性置换不变性。考虑一层的输出

$$x = \max_{j: (i, j) \in E} h(x, x) \quad (10)$$

以及一个排列算子 π 。该层的输出 x 对输入 x 的排列保持不变，因为 $\max$ 是一个对称函数（其他对称函数同样适用）。用于聚合点特征的的全局最大池化算子也是排列不变的。

平移不变性。我们的算子具有“部分”平移不变性，即我们选择的边函数（7）明确地暴露了函数中可以依赖于平移的部分，并且可以选择禁用。考虑对 x 和 x 应用平移；我们可以证明当平移 T 时，部分边特征得以保留。特别是，对于平移后的点云我们有

$$e = \theta \cdot (x + T - (x + T)) + \phi \cdot (x + T) = \theta \cdot (x - x) + \phi \cdot (x + T).$$

如果我们仅通过设置 $\phi = 0$ 来考虑 $x - x$ ，那么该算子对平移完全不变。然而，在这种情况下，模型简化为基于无序的补丁集来识别物体，忽略了补丁的位置和方向。同时输入 $x - x$ 和 x 时，模型考虑了补丁的局部几何形状，同时保留了全局形状信息。

3.4 与现有方法的比较

DGCNN 与两种方法类别相关，即 PointNet 和图 CNN，我们证明它们是我们方法的特定设置。我们在表 1 中总结了不同方法。

PointNet 是我们方法的一个特例，其中 $k=1$ ，生成一个边集为空 $E=\emptyset$ 的图。PointNet 使用的边函数是 $h(x, x) = h(x)$ ，它考虑全局但不考虑局部几何。PointNet++ 试图通过以局部方式应用 PointNet 来考虑局部结构。在我们的术语中，PointNet++ 首先根据点之间的欧几里得距离构建图，并在每一层应用图粗化操作。对于每一层，使用最远点采样（FPS）选择一些点；仅保留选中的点，而其他点在经过这一层后直接丢弃。通过这种方式，每层操作后图都会变得更小。与 DGCNN 不同，PointNet++ 使用点输入坐标计算成对距离，因此它们的图在训练期间是固定的。PointNet++ 使用的边函数是 $h(x, x) = h(x)$ ，聚合操作也是 $\max$ 。

在图卷积网络中，MoNet [Monti 等人 2017a]、ECC [Simonovsky 和 Komodakis 2017]、图注意力网络 [Velickovic 等人 2017] 以及同期工作 [Atzmon 等人 2018] 是最相关的方法。它们的共同点是图上的局部块的概念，其中可以定义卷积型操作。

具体来说，Monti 等人 [2017a] 利用图结构计算局部“伪坐标系” u ，其中邻域顶点被表示；然后卷积被定义为 M 分量高斯混合

$$x = \bigoplus_{j: (i, j) \in E} \theta \cdot (x \odot \varphi(u(x, x))), \quad (11)$$

其中 φ 是一个高斯核， $\odot$ 是逐元素（Hadamard）乘积， $\{w, \dots, w\}$ 编码高斯的可学习参数（均值和协方差），而 $\{\theta, \dots, \theta\}$ 是可学习的滤波器系数。(11) 是我们通用操作(1)的一个实例，其中

²[Simonovsky and Komodakis 2017; Velickovic et al. 2017] 可以视为 [Monti et al. 2017a] 的实例，不同之处在于权重是使用相邻节点的特征而不是图结构构建的；[Atzmon et al. 2018] 也相似，只是权重函数是手工设计的。

	Aggregation	Edge Function	Learnable parameters
PointNet [Qi et al. 2017b]	—	$h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = h_{\Theta}(\mathbf{x}_i)$	Θ
PointNet++ [Qi et al. 2017c]	max	$h_{\Theta}(\mathbf{x}_i, \mathbf{x}_j) = h_{\Theta}(\mathbf{x}_j)$	Θ
MoNet [Monti et al. 2017a]	$\sum$	$h_{\theta_m, \mathbf{w}_n}(\mathbf{x}_i, \mathbf{x}_j) = \theta_m \cdot (\mathbf{x}_j \odot g_{\mathbf{w}_n}(u(\mathbf{x}_i, \mathbf{x}_j)))$	$\mathbf{w}_n, \theta_m$
PCNN [Atzmon et al. 2018]	$\sum$	$h_{\theta_m}(\mathbf{x}_i, \mathbf{x}_j) = (\theta_m \cdot \mathbf{x}_j)g(u(\mathbf{x}_i, \mathbf{x}_j))$	θ_m

Table 1. Comparison to existing methods. The per-point weight w_i in [Atzmon et al. 2018] effectively is computed in the first layer and could be carried onward as an extra feature; we omit this for simplicity.

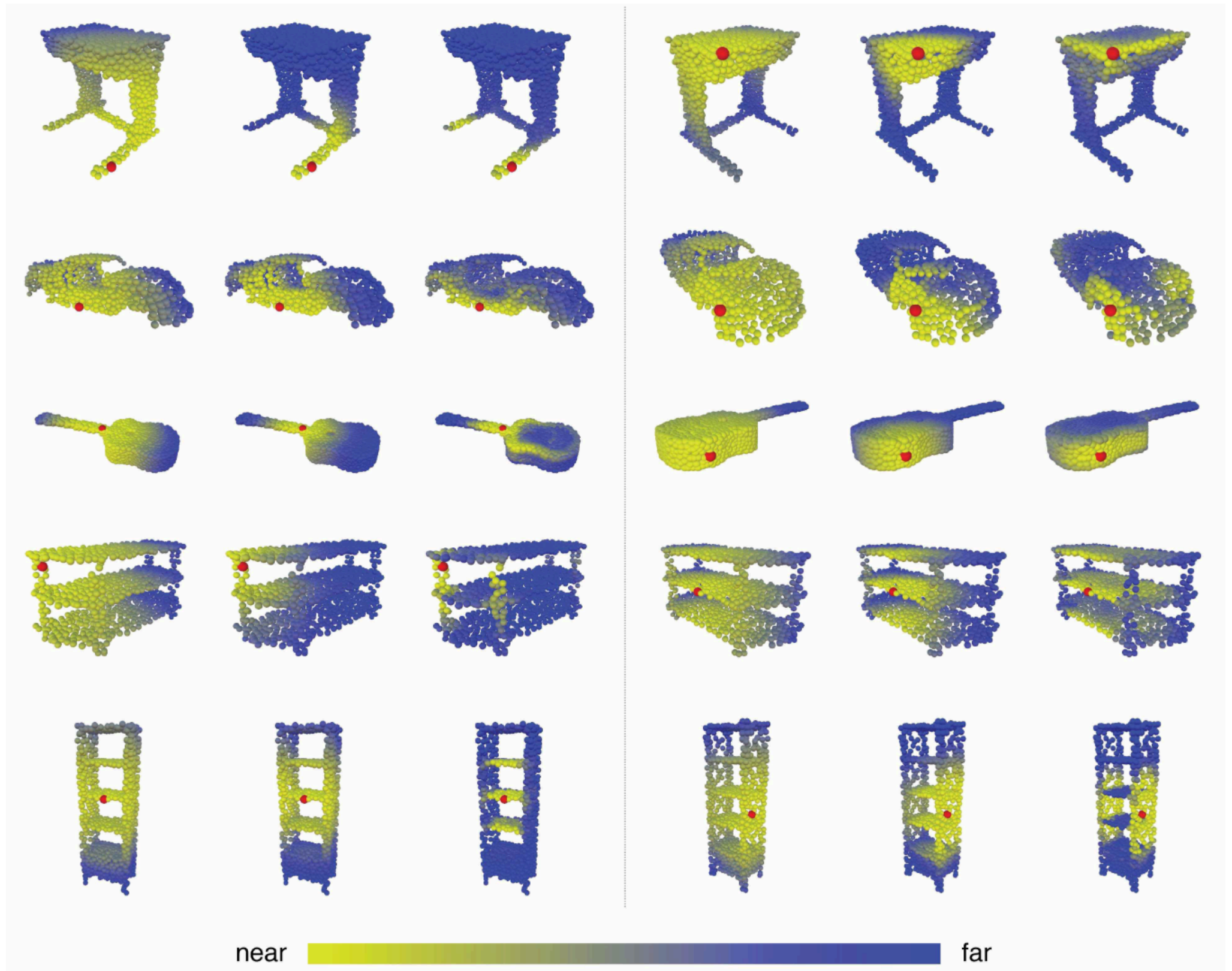


Fig. 4. **Structure of the feature spaces** produced at different stages of our shape classification neural network architecture, visualized as the distance between the red point to the rest of the points. For each set, **Left**: Euclidean distance in the input $\mathbb{R}^3$ space; **Middle**: Distance after the point cloud transform stage, amounting to a global transformation of the shape; **Right**: Distance in the feature space of the last layer. Observe how in the feature space of deeper layers semantically similar structures such as shelves of a bookshelf or legs of a table are brought close together, although they are distant in the original space.

聚合边函数可学习参数 PointNet [Qi 等人 2017b] — $h_{\Theta}(x_i, x_j) = h_{\Theta}(x_i) \ominus \text{PointNet++ [Qi 等人 2017c]} \max h_{\Theta}(x_i, x_j)$
= $h_{\Theta}(x_j) \ominus \text{MoNet [Monti 等人 2017a]} \mid h_{\Theta, w}(x_i, x_j) = \theta_m \cdot (x_j \odot \text{dw}(u(x_i, x_j))) \text{wn}, \theta_m \text{ PCNN [Atzmon 等人 2018]} \mid h_{\Theta}(x_i, x_j) = (\theta_m \cdot x_j) \text{d}(u(x_i, x_j)) \ominus \theta_m$

表 1. 与现有方法的对比。每点权重 w_i [Atzmon 等人 2018] 实际上在第一层有效计算，并可作为额外特征继续传递；我们为简化起见省略了这一点。

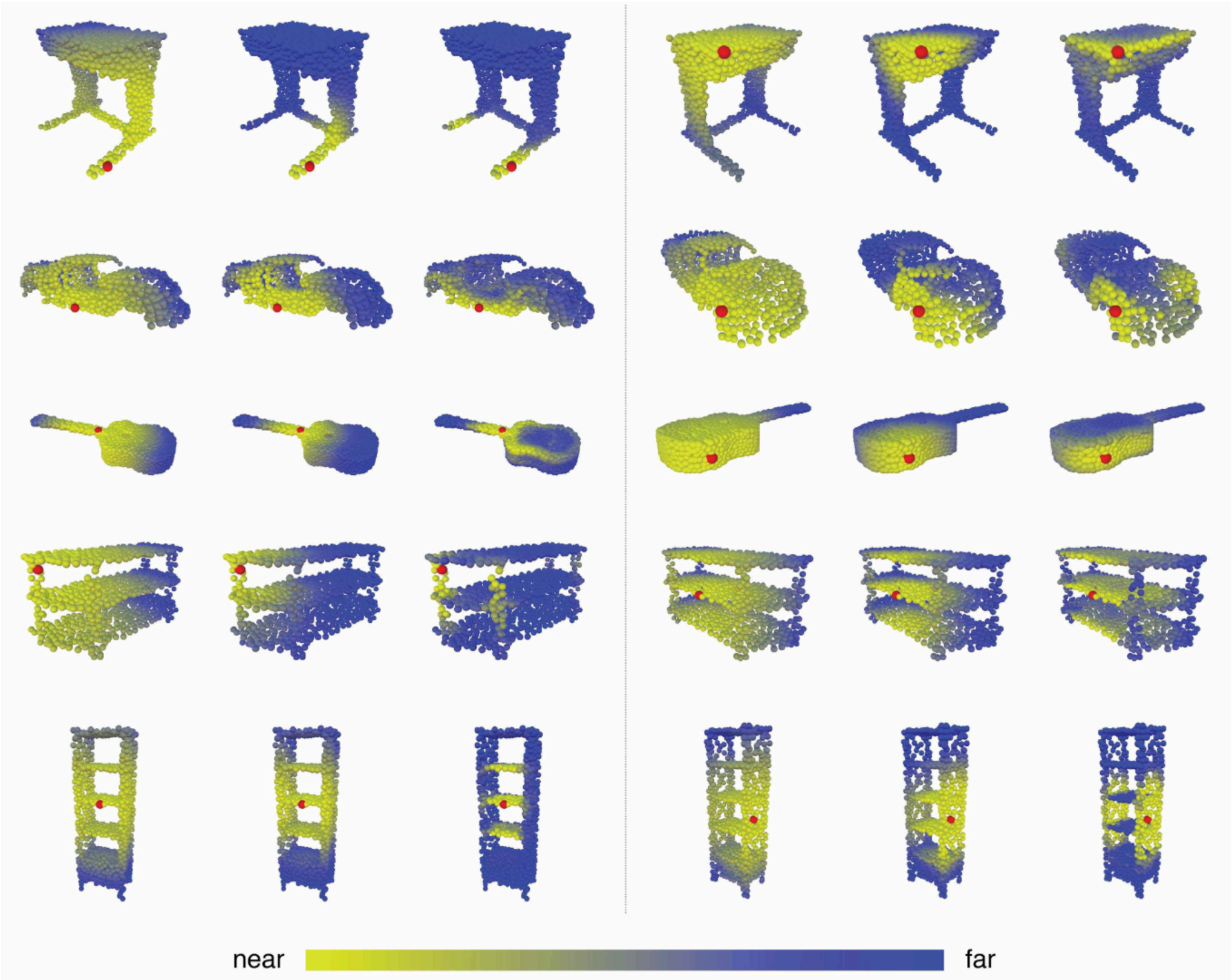


图 4. 我们形状分类神经网络架构不同阶段产生的特征空间结构，以红色点到其余点的距离可视化。对于每个集合，左侧：输入 R 空间的欧几里得距离；中间：点云变换阶段的距离，相当于形状的全局变换；右侧：最后一层特征空间中的距离。观察深层特征空间中，语义相似的结构距离如何拉近，如书架的搁板或桌子的腿，尽管它们在原始空间中距离较远。

particular edge function

$$h_{\theta_m, w_n}(x_i, x_j) = \theta_m \cdot (x_j \odot g_{w_n}(u(x_i, x_j)))$$

and $\square = \sum$. Again, their graph structure is fixed, and u is constructed based on the degrees of nodes.

[Atzmon et al. 2018] can be seen as a special case of [Monti et al. 2017a] with g as predefined Gaussian functions. Removing learnable parameters ($w_1, \dots, w_N$) and constructing a dense graph from point clouds, we have

$$x'_{im} = \sum_{j:j \in \mathcal{V}} (\theta_m \cdot x_j) g(u(x_i, x_j)), \quad (12)$$

where u is the pairwise distance between x_i and x_j in Euclidean space.

While MoNet and other graph CNNs assume a given fixed graph on which convolution-like operations are applied, to our knowledge our method is the first for which the graph changes from layer to layer and even on the same input during training when learnable parameters are updated. This way, our model not only learns how to extract local geometric features, but also how to group points in a point cloud. Figure 4 shows the distance in different feature spaces, exemplifying that the distances in deeper layers carry semantic information over long distances in the original embedding.

4 EVALUATION

In this section, we evaluate the models constructed using EdgeConv for different tasks: classification, part segmentation, and semantic segmentation. We also visualize experimental results to illustrate key differences from previous work.

4.1 Classification

Data. We evaluate our model on the ModelNet40 [Wu et al. 2015] classification task, consisting in predicting the category of a previously unseen shape. The dataset contains 12,311 meshed CAD models from 40 categories. 9,843 models are used for training and 2,468 models are for testing. We follow verbatim the experimental settings of Qi et al. [2017b]. For each model, 1,024 points are uniformly sampled from the mesh faces; the point cloud is rescaled to fit into the unit sphere. Only the (x, y, z) coordinates of the sampled points are used, and the original meshes are discarded. During the training procedure, we augment the data by randomly scaling objects and perturbing the object and point locations.

Architecture. The network architecture used for the classification task is shown in Figure 3 (top branch without spatial transformer network). We use four EdgeConv layers to extract geometric features. The four EdgeConv layers use three shared fully-connected layers (64, 64, 128, 256). We recompute the graph based on the features of each EdgeConv layer and use the new graph for next layer. The number k of nearest neighbors is 20 for all EdgeConv layers (for the last row in Table 2, k is 40). Shortcut connections are included to extract multi-scale features and one shared fully-connected layer (1024) to aggregate multi-scale features, where we concatenate features from previous layers to get a $64+64+128+256=512$ dimensional point cloud. Then, a global max/sum pooling is used to get the point cloud global feature, after which two fully-connected layers (512, 256) are used to transform the global feature. Dropout with

keep probability of 0.5 is used in the last two fully-connected layers. All layers include LeakyReLU and batch normalization. The number k was chosen using a validation set. We split the training data to 80% for training and 20% for validation to search the best k . After k is chosen, we retrain the model on the whole training data and evaluate the model on the testing data. Other hyperparameters were chosen in a similar ways.

Training. We use SGD with learning rate 0.1, and we reduce the learning rate until 0.001 using cosine annealing [Loshchilov and Hutter 2017]. The momentum for batch normalization is 0.9, and we do not use batch normalization decay. The batch size is 32 and the momentum is 0.9.

Results. Table 2 shows the results for the classification task. Our model achieves the best results on this dataset. Our baseline using a fixed graph determined by proximity in the input point cloud is 1.0% better than PointNet++. An advanced version including dynamical graph recomputation achieves the best results on this dataset. All the experiments are performed with point clouds that contain 1024 points except last row. We further test out model with 2048 points. The k used for 2048 points is 40 to maintain the same density. Note that PCNN [Atzmon et al. 2018] uses additional augmentation techniques like randomly sampling 1024 points out of 1200 points during both training and testing.

	MEAN CLASS ACCURACY	OVERALL ACCURACY
3DShapeNets [Wu et al. 2015]	77.3	84.7
VoxNet [Maturana and Scherer 2015]	83.0	85.9
SubVolume [Qi et al. 2016]	86.0	89.2
VRN (Single View) [Brock et al. 2016]	88.98	-
VRN (Multiple Views) [Brock et al. 2016]	91.33	-
ECC [Simonovsky and Komodakis 2017]	83.2	87.4
PointNet [Qi et al. 2017b]	86.0	89.2
PointNet++ [Qi et al. 2017c]	-	90.7
KD-Net [Klokov and Lempitsky 2017]	-	90.6
PointCNN [Li et al. 2018a]	88.1	92.2
PCNN [Atzmon et al. 2018]	-	92.3
Ours (Baseline)	88.9	91.7
Ours	90.2	92.9
Ours (2048 points)	90.7	93.5

Table 2. Classification results on ModelNet40.

4.2 Model Complexity

We use the ModelNet40 [Wu et al. 2015] classification experiment to compare the complexity of our model to previous state-of-the-art. Table 3 shows that our model achieves the best tradeoff between the model complexity (number of parameters), computational complexity (measured as forward pass time), and the resulting classification accuracy.

Our baseline model using the fixed k -NN graph outperforms the previous state-of-the-art PointNet++ by 1.0% accuracy, at the same time being 7 times faster. A more advanced version of our model including a dynamically-updated graph computation outperforms PointNet++, PCNN by 2.2% and 0.6% respectively, while being much

特定的边函数

$$h(x, x) = \theta \cdot (x \odot d(u(x, x)))$$

再次，它们的图结构是固定的，u 是根据节点的度数构建的。

[Atzmon 等人. 2018] 可以看作是 [Monti 等人. 2017a] 的一个特例，其中 d 是预定义的高斯函数。移除可学习参数 $(w, \dots, w)$ 并从点云中构建密集图，我们得到

$$x = \frac{1}{\sum_{j:j \in V} (\theta \cdot x) d(u(x, x))}, (12)$$

其中 u 是 x 和 x 在欧几里得空间中的成对距离。

虽然 MoNet 和其他图卷积神经网络假设一个固定的图，并在其上应用卷积类操作，但据我们所知，我们的方法是目前首个在层与层之间以及训练过程中同一输入上，当可学习参数更新时图会发生变化的方法。通过这种方式，我们的模型不仅学习如何提取局部几何特征，还学习如何对点云中的点进行分组。图 4 展示了不同特征空间中的距离，举例说明深层中的距离在原始嵌入中携带了长距离的语义信息。

4 评估

在本节中，我们评估使用 EdgeConv 构建的模型在不同任务上的表现：分类、部件分割和语义分割。我们还可可视化实验结果，以说明与先前工作的关键差异。

4.1 分类数据。我们在 ModelNet40 [Wu 等人, 2015] 分类任务上评估我们的模型，该任务包括预测先前未见过的形状的类别。该数据集包含来自 40 个类别的 12,311 个网格 CAD 模型。9,843 个模型用于训练，2,468 个模型用于测试。我们完全遵循 Qi 等人[2017b]的实验设置。对于每个模型，从网格面中均匀采样 1,024 个点；将点云缩放到适合单位球。仅使用采样点的 (x, y, z) 坐标，并丢弃原始网格。在训练过程中，我们通过随机缩放对象和扰动对象和点位置来增强数据。

架构。用于分类任务的网络架构如图 3 所示（不包括空间变换网络的上分支）。我们使用四个 EdgeConv 层来提取几何特征。这四个 EdgeConv 层使用三个共享的全连接层（64, 64, 128, 256）。我们根据每个 EdgeConv 层的特征重新计算图，并使用新图进行下一层。所有 EdgeConv 层的最近邻数量 k 为 20（对于表 2 中的最后一行， k 为 40）。我们包含快捷连接来提取多尺度特征，并使用一个共享的全连接层（1024）来聚合多尺度特征，其中我们将前几层的特征连接起来得到一个 $64+64+128+256=512$ 维的点云。然后，使用全局最大/求和池化来获取点云全局特征，之后使用两个全连接层（512, 256）来转换全局特征。

在最后两个全连接层使用了保持概率为 0.5 的 Dropout。所有层都包含 LeakyReLU 和批量归一化。通过验证集选择了数值 k 。我们将训练数据分为 80%用于训练和 20%用于验证，以搜索最佳 k 值。选择 k 值后，我们在整个训练数据上重新训练模型，并在测试数据上评估模型。其他超参数也是以类似方式选择的。

训练。我们使用学习率为 0.1 的 SGD，并使用余弦退火将学习率降低至 0.001 [Loshchilov and Hutter 2017]。批量归一化的动量为 0.9，我们不使用批量归一化衰减。批大小为 32，动量为 0.9。

结果。表 2 显示了分类任务的结果。我们的模型在这个数据集上取得了最佳结果。我们的基线模型使用输入点云中根据邻近关系确定的固定图，比 PointNet++好 1.0%。一个包括动态图重新计算的改进版本在这个数据集上取得了最佳结果。除了最后一行，所有实验都是使用包含 1024 个点的点云进行的。我们进一步用 2048 个点测试了我们的模型。2048 个点使用的 k 值是 40，以保持相同的密度。请注意，PCNN [Atzmon et al. 2018] 使用了额外的增强技术，比如在训练和测试过程中随机从 1200 个点中采样 1024 个点。

总体类别准确率	准确率	3DShapeNets [Wu 等人 2015]	77.3	84.7	VoxNet [Maturana 和 Scherer 2015]	83.0	85.9	Subvolume [Qi 等人 2016]	86.0	89.2	VRN（单视图）[Brock 等人 2016]	88.98	VRN（多视图）[Brock 等人 2016]	91.33	ECC [Simonovsky 和 Komodakis 2017]	83.2	87.4	PointNet [Qi 等人 2017b]	86.0	89.2	PointNet++ [Qi 等人 2017c]	-	90.7	Kd-net [Klokov 和 Lempitsky 2017]	-	90.6	PointCNN [Li 等人 2018a]	88.1	92.2	PCNN [Atzmon 等人 2018]	-	92.3	我们（基线）	88.9	91.7
---------	-----	--------------------------	------	------	----------------------------------	------	------	------------------------	------	------	-------------------------	-------	-------------------------	-------	-----------------------------------	------	------	------------------------	------	------	--------------------------	---	------	----------------------------------	---	------	------------------------	------	------	-----------------------	---	------	--------	------	------

我们	90.2	92.9
我们（2048 个点）	90.7	93.5

表 2. ModelNet40 上的分类结果

4.2 模型复杂度

我们使用 ModelNet40 [Wu 等人 2015] 分类实验来比较我们的模型与先前最先进技术的复杂度。表 3 显示，我们的模型在模型复杂度（参数数量）、计算复杂度（以正向传递时间衡量）以及最终分类精度之间实现了最佳权衡。

我们的基准模型使用固定 k -NN 图，在精度上比先前最先进的 PointNet++提高了 1.0%，同时速度提高了 7 倍。我们模型的更高级版本，包括动态更新的图计算，在精度上分别比 PointNet++和 PCNN 提高了 2.2%和 0.6%，同时速度也大幅

	MODEL SIZE(MB)	TIME(MS)	ACCURACY(%)
POINTNET (BASELINE) [QI ET AL. 2017B]	9.4	6.8	87.1
POINTNET [QI ET AL. 2017B]	40	16.6	89.2
POINTNET++ [QI ET AL. 2017C]	12	163.2	90.7
PCNN [ATZMON ET AL. 2018]	94	117.0	92.3
OURS (BASELINE)	11	19.7	91.7
OURS	21	27.2	92.9

Table 3. Complexity, forward time, and accuracy of different models

more efficient. The number of points in each experiment is also 1024 in this section.

4.3 More Experiments on ModelNet40

We also experiment with various settings of our model on the ModelNet40 [Wu et al. 2015] dataset. In particular, we analyze the effectiveness of the different distance metrics, explicit usage of $\mathbf{x}_i - \mathbf{x}_j$, and more points.

Table 4 shows the results. “Centralization” denotes using concatenation of $\mathbf{x}_i$ and $\mathbf{x}_i - \mathbf{x}_j$ as the edge features rather than concatenating $\mathbf{x}_i$ and $\mathbf{x}_j$. “Dynamic graph recomputation” denotes we reconstruct the graph rather than using a fixed graph. Explicitly centralizing each patch by using the concatenation of $\mathbf{x}_i$ and $\mathbf{x}_i - \mathbf{x}_j$ leads to about 0.5% improvement for overall accuracy. By dynamically updating graph, there is about 0.7% improvement, and Figure 4 also suggests that the model can extract semantically meaningful features. Using more points further improves the overall accuracy by 0.6%.

We also experiment with different numbers k of nearest neighbors as shown in Table 5. For all experiments, the number of points is still 1024. While we do not exhaustively experiment with all possible k , we find with large k that the performance degenerates. This confirms our hypothesis that for certain density, with large k the Euclidean distance fails to approximate geodesic distance, destroying the geometry of each patch.

We further evaluate the robustness of our model (trained on 1,024 points with $k = 20$) to point cloud density. We simulate the environment that random input points drops out during testing. Figure 5 shows that even half of points is dropped, the model still achieves reasonable results. With fewer than 512 points, however, performance degenerates dramatically.

CENT	DYN	MPOINTS	MEAN CLASS ACCURACY(%)	OVERALL ACCURACY(%)
			88.9	91.7
x			89.3	92.2
x	x		90.2	92.9
x	x	x	90.7	93.5

Table 4. Effectiveness of different components. CENT denotes centralization, DYN denotes dynamical graph recomputation, and MPOINTS denotes experiments with 2048 points

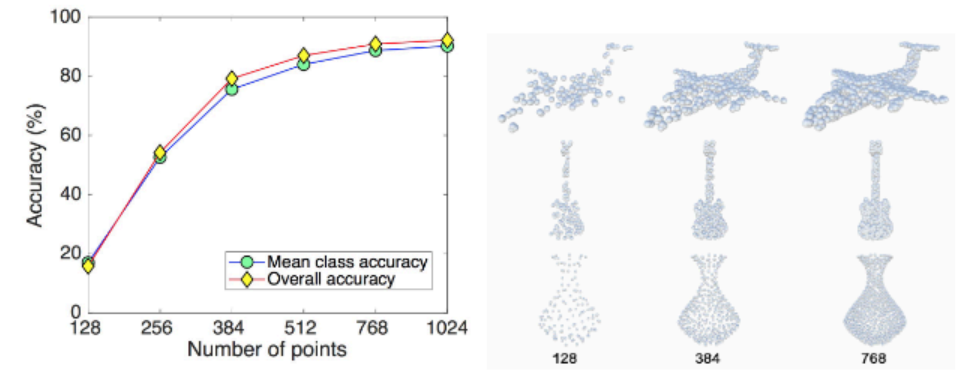


Fig. 5. **Left:** Results of our model tested with random input dropout. The model is trained with number of points being 1024 and k being 20. **Right:** Point clouds with different number of points. The numbers of points are shown below the bottom row.

NUMBER OF NEAREST NEIGHBORS (k)	MEAN CLASS ACCURACY(%)	OVERALL ACCURACY(%)
5	88.0	90.5
10	88.9	91.4
20	90.2	92.9
40	89.4	92.4

Table 5. Results of our model with different numbers of nearest neighbors.

4.4 Part Segmentation

Data. We extend our EdgeConv model architectures for part segmentation task on ShapeNet part dataset [Yi et al. 2016]. For this task, each point from a point cloud set is classified into one of a few predefined part category labels. The dataset contains 16,881 3D shapes from 16 object categories, annotated with 50 parts in total. 2,048 points are sampled from each training shape, and most sampled point sets are labeled with less than six parts. We follow the official train/validation/test split scheme as Chang et al. [2015] in our experiment.

Architecture. The network architecture is illustrated in Figure 3 (bottom branch). After a spatial transformer network, three EdgeConv layers are used. A shared fully-connected layer (1024) aggregates information from the previous layers. Shortcut connections are used to include all the EdgeConv outputs as local feature descriptors. At last, three shared fully-connected layers (256, 256, 128) are used to transform the pointwise features. Batch-norm, dropout, and ReLU are included in the similar fashion to our classification network.

Training. The same training setting as in our classification task is adopted. A distributed training scheme is further implemented on two NVIDIA TITAN X GPUs to maintain the training batch size.

Results. We use Intersection-over-Union (IoU) on points to evaluate our model and compare with other benchmarks. We follow the same evaluation scheme as PointNet: The IoU of a shape is computed by averaging the IoUs of different parts occurring in that shape, and the IoU of a category is obtained by averaging the IoUs of all the shapes belonging to that category. The mean IoU (mIoU) is finally calculated by averaging the IoUs of all the testing shapes. We

模型大小(MB)	时间(ms)	准确率(%)	PointNet (基线)	[Qi 等人 2017b]	9.4	6.8	87.1	PointNet [Qi 等人 2017b]	40	16.6	89.2	PointNet++ [Qi 等人 2017c]	12	163.2	90.7	PCNN [Atzmon 等人 2018]	94	117.0
92.3	我们 (基线)	11	19.7	91.7	我们	21	27.2	92.9										

表 3. 不同模型的复杂度、前向时间和准确率

更高效。本节中每个实验的点数也是 1024。

4.3 在 ModelNet40 上的更多实验

我们在 ModelNet40 [Wu et al. 2015]数据集上对模型的多种设置进行了实验。具体而言，我们分析了不同距离度量、显式使用 $x-x$ 以及更多点数的效果。

表 4 展示了结果。“Centralization”表示使用 x 和 $x-x$ 的拼接作为边特征，而不是拼接 x 和 x 。“Dynamic graph recomputation”表示我们重建图，而不是使用固定图。通过使用 x 和 $x-x$ 的拼接显式地集中每个补丁，整体准确率提高了约 0.5%。通过动态更新图，准确率提高了约 0.7%，图 4 也表明模型可以提取语义上有意义的特征。使用更多点数进一步将整体准确率提高了 0.6%。

我们也实验了不同的最近邻数量 k ，如表 5 所示。所有实验中，点的数量仍然是 1024。虽然我们没有对所有可能的 k 进行穷尽实验，但我们发现当 k 较大时，性能会退化。这证实了我们的假设：对于一定的密度，当 k 较大时，欧几里得距离无法近似测地距离，从而破坏了每个补丁的几何结构。

我们进一步评估了我们的模型（在 1024 个点、 $k=20$ 的情况下训练）对点云密度的鲁棒性。我们模拟了在测试过程中随机输入点会丢失的环境。图 5 显示，即使一半的点丢失，模型仍然能取得合理的结果。然而，当点数少于 512 时，性能会急剧退化。

CENT	DYN	MPOINTS	Mean Class Accuracy(%)	Overall Accuracy(%)
			88.9	91.7
x			89.3	92.2
x	x		90.2	92.9
x	x	x	90.7	93.5

表 4. 不同组件的有效性。CENT 表示中心化，DYN 表示动态图重新计算，MPOINTS 表示使用 2048 个点的实验

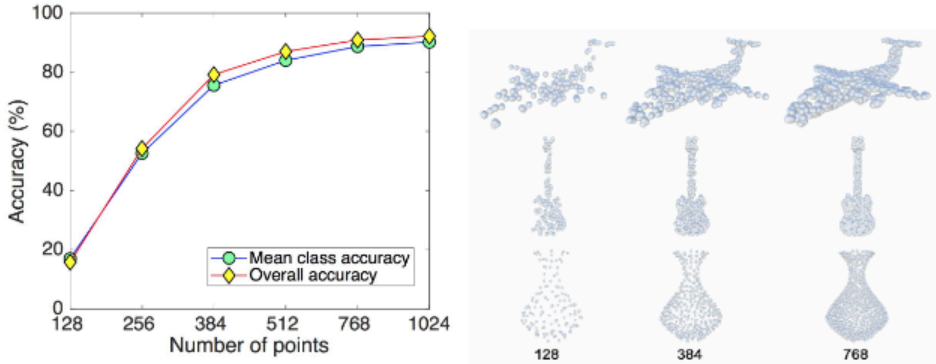


图 5. 左：使用随机输入 dropout 测试的模型结果。模型使用 1024 个点进行训练， k 值为 20。右：具有不同点数的点云。点数显示在底部一行的下方。

最近邻数量 (k)	平均总体	类别	准确率(%)	准确率(%)
5	88.0	90.5		
10	88.9	91.4		
20	90.2	92.9		
40	89.4	92.4		

表 5. 模型使用不同最近邻数量的结果。

4.4 部分分割数据。我们扩展了 EdgeConv 模型架构，用于在 ShapeNet 部分数据集[Yi 等人, 2016]上的部分分割任务。对于此任务，点云集中的每个点被分类为预定义的部分类别标签之一。该数据集包含来自 16 个物体类别的 16,881 个 3D 形状，总共标注了 50 个部分。从每个训练形状中采样 2,048 个点，大多数采样点集标注的部分少于六个。我们在实验中遵循 Chang 等人[2015]的官方训练/验证/测试划分方案。

架构。网络架构如图 3（底部分支）所示。经过一个空间变换网络后，使用三个 EdgeConv 层。一个共享的全连接层（1024）用于聚合前几层的信息。使用快捷连接将所有 EdgeConv 输出作为局部特征描述符。最后，使用三个共享的全连接层（256, 256, 128）来转换点特征。与我们的分类网络类似，包括批归一化、dropout 和 ReLU。训练。我们采用了与分类任务相同的训练设置。进一步在两个 NVIDIA TITAN X GPU 上实现了分布式训练方案，以保持训练批次大小。

结果。我们使用点上的交并比（IoU）来评估我们的模型，并与其他基准进行比较。我们遵循与 PointNet 相同的评估方案：形状的 IoU 是通过平均该形状中出现的不同部分的 IoU 来计算的，而类别的 IoU 是通过平均属于该类别的所有形状的 IoU 来获得的。最后，通过平均所有测试形状的 IoU 来计算平均 IoU（mIoU）。我们

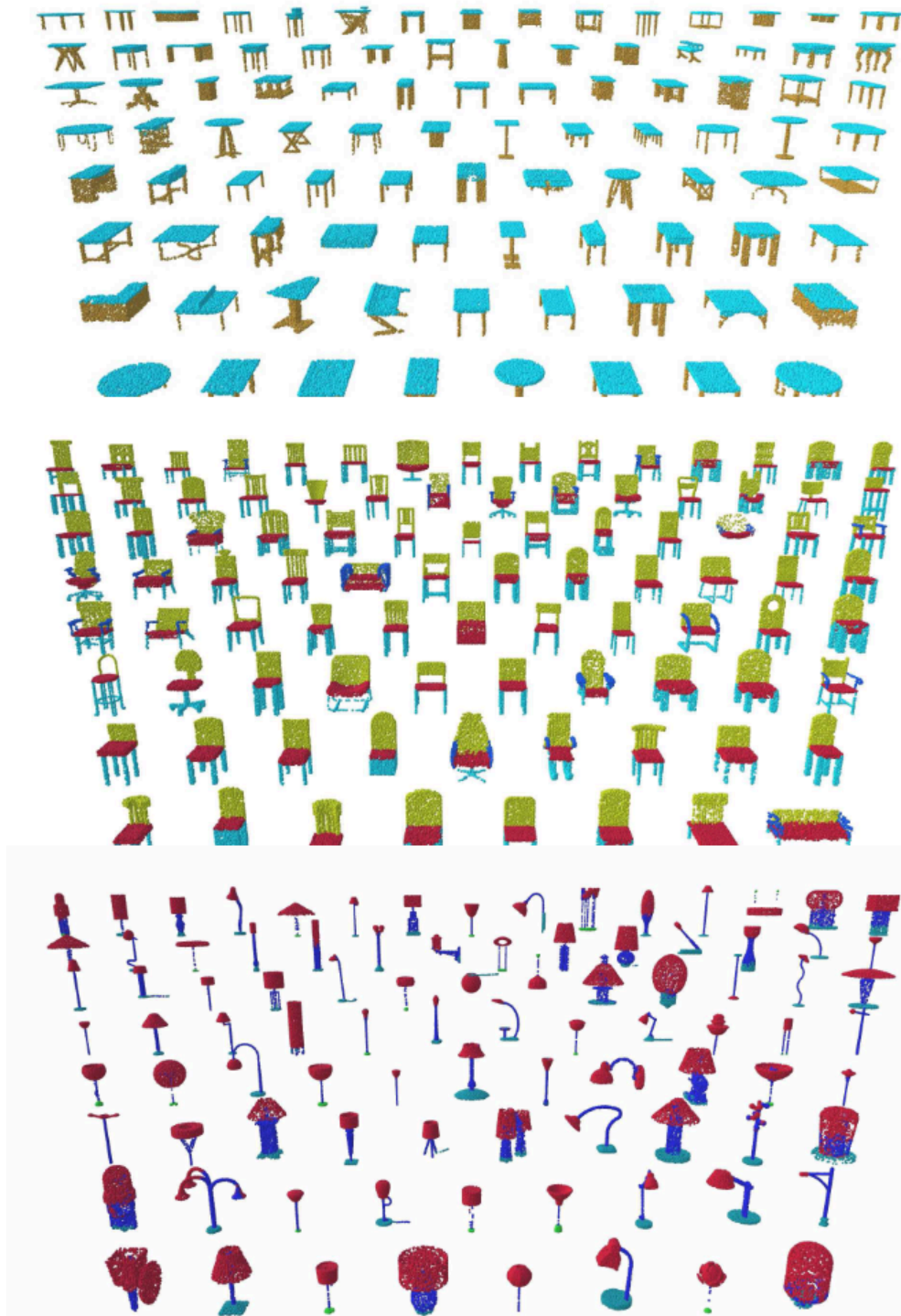


Fig. 6. Our part segmentation testing results for tables, chairs and lamps.

compare our results with PointNet [Qi et al. 2017b], PointNet++ [Qi et al. 2017c], Kd-Net [Klokov and Lempitsky 2017], LocalFeatureNet [Shen et al. 2017], PCNN [Atzmon et al. 2018], and PointCNN [Li et al. 2018a]. The evaluation results are shown in Table 6. We also visually compare the results of our model and PointNet in Figure 7. More examples are shown in Figure 6.

Intra-cloud distances. We next explore the relationships between different point clouds captured using our features. As shown in Figure 8, we take one red point from a source point cloud and compute its distance in feature space to points in other point clouds from the same category. An interesting finding is that although points are from different sources, they are close to each other if they are from semantically similar parts. We evaluate on the features after the third layer of our segmentation model for this experiment.

Segmentation on partial data. Our model is robust to partial data. We simulate the environment that part of the shape is dropped from one of six sides (top, bottom, right, left, front and back) with different percentages. The results are shown in Figure 9. On the left,

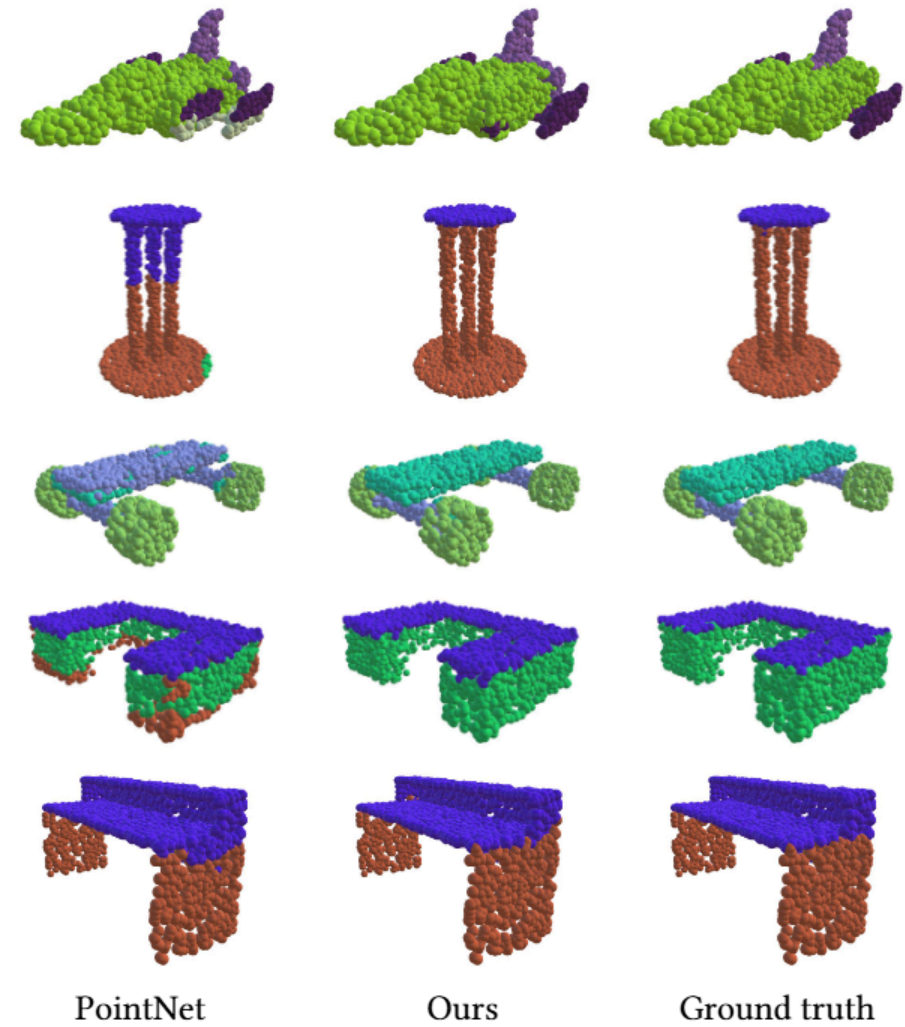


Fig. 7. Compare part segmentation results. For each set, from left to right: PointNet, ours and ground truth.

the mean IoU versus “keep ratio” is shown. On the right, the results for an airplane model are visualized.

4.5 Indoor Scene Segmentation

Data. We evaluate our model on Stanford Large-Scale 3D Indoor Spaces Dataset (S3DIS) [Armeni et al. 2016] for a semantic scene segmentation task. This dataset includes 3D scan point clouds for 6 indoor areas including 272 rooms in total. Each point belongs to one of 13 semantic categories—e.g. board, bookcase, chair, ceiling, and beam—plus clutter. We follow the same setting as Qi et al. [2017b], where each room is split into blocks with area $1m \times 1m$, and each point is represented as a 9D vector (XYZ, RGB, and normalized spatial coordinates). 4,096 points are sampled for each block during training process, and all points are used for testing. We also use the same 6-fold cross validation over the 6 areas, and the average evaluation results are reported.

The model used for this task is similar to part segmentation model, except that a probability distribution over semantic object classes is generated for each input point and no categorical vector is used here. We compare our model with both PointNet [Qi et al. 2017b] and PointNet baseline, where additional point features (local point density, local curvature and normal) are used to construct handcrafted features and then fed to an MLP classifier. We further compare our work with [Engelmann et al. 2017] and PointCNN [Li et al. 2018a]. Engelmann et al. [2017] present network architectures to enlarge the receptive field over the 3D scene. Two different approaches are proposed in their work: MS+CU for multi-scale block features with consolidation units; G+RCU for the grid-blocks with recurrent

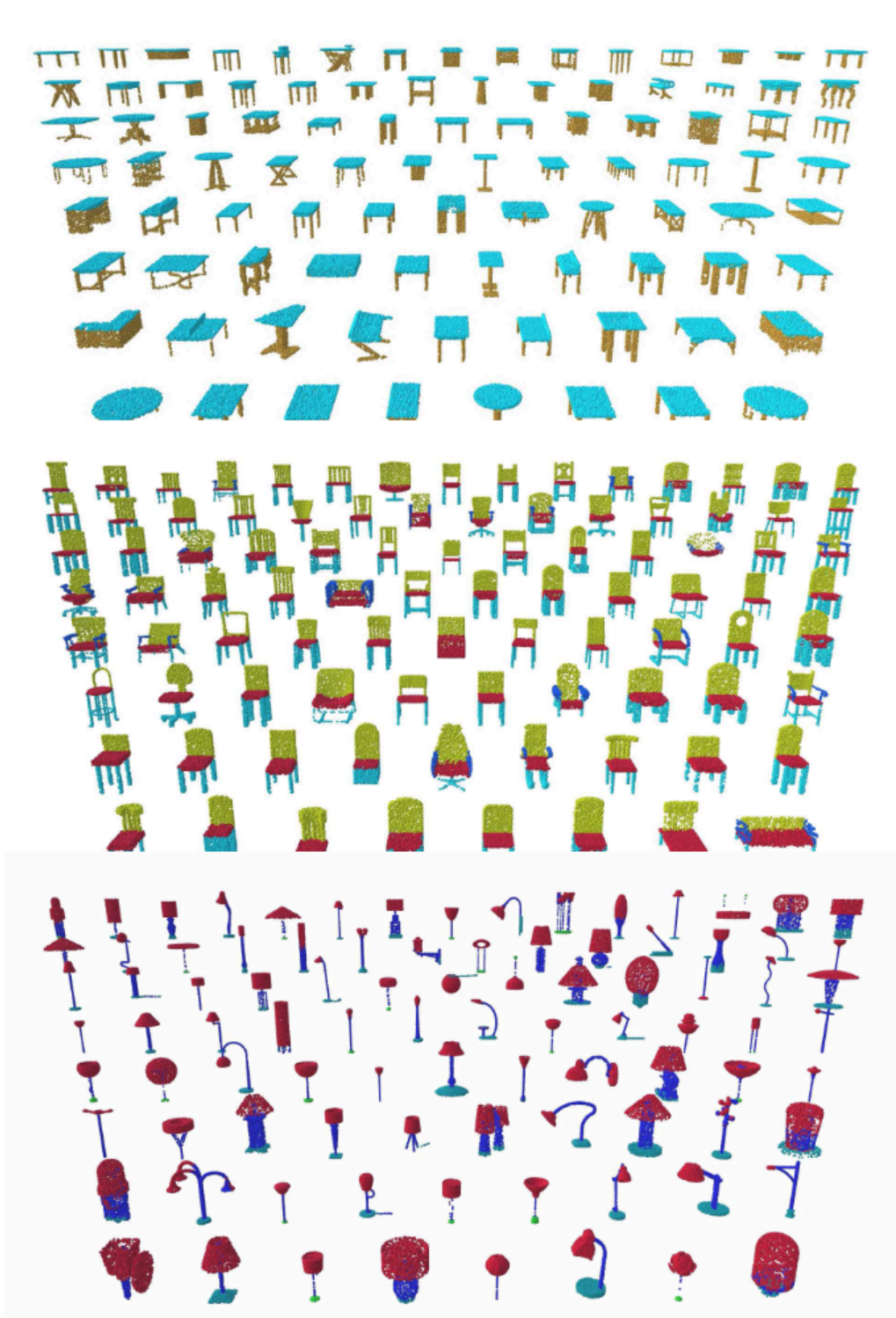


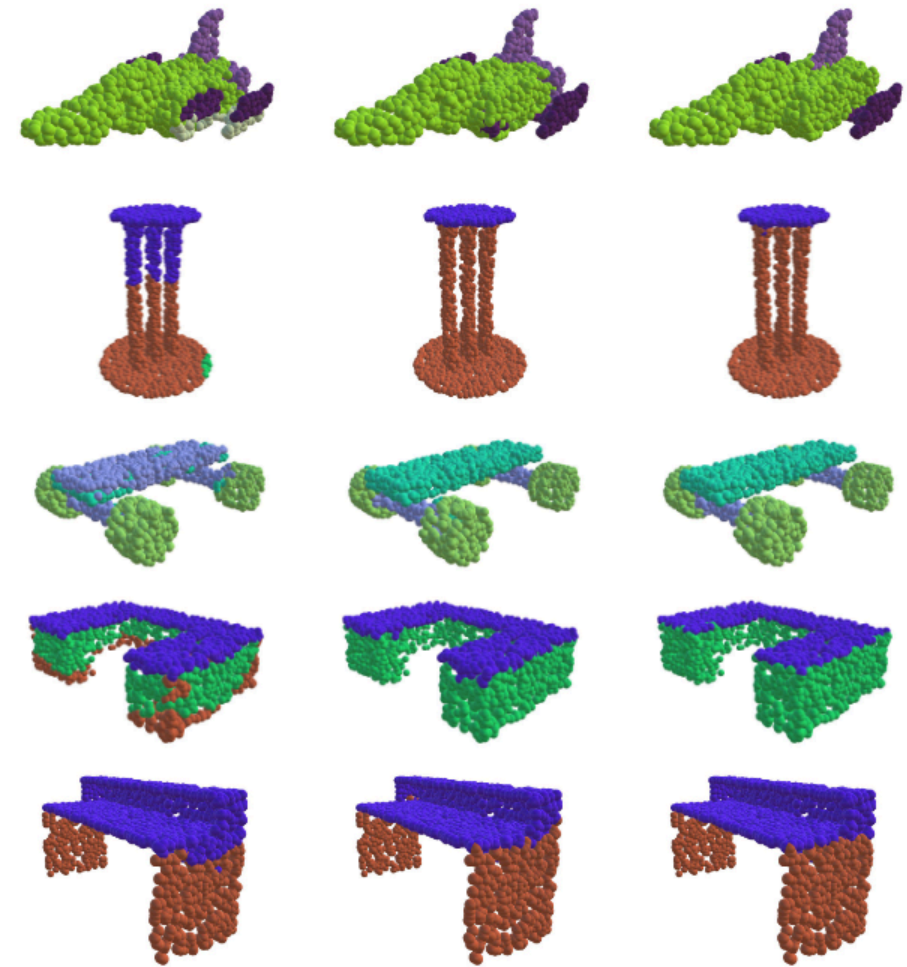
图 6. 我们对桌子、椅子和灯的部分分割测试结果。

将我们的结果与 PointNet [Qi 等人 2017b]、PointNet++ [Qi 等人 2017c]、Kd-Net [Klokov 和 Lempitsky 2017]、LocalFeatureNet [Shen 等人 2017]、PCNN [Atzmon 等人 2018] 和 PointCNN [Li 等人 2018a] 进行比较。评估结果如表 6 所示。我们还在图 7 中直观地比较了我们的模型和 PointNet 的结果。

更多示例显示在图 6 中。

云内距离。接下来，我们探索使用我们的特征捕获的不同点云之间的关系。如图 8 所示，我们从源点云中取一个红点，并计算它在特征空间中到同一类别其他点云中点的距离。一个有趣的发现是，尽管点来自不同的源，但如果它们来自语义上相似的部分，它们彼此靠近。我们在这个实验中评估我们分割模型的第三层特征。

部分数据上的分割。我们的模型对部分数据具有鲁棒性。我们模拟了从六个方向（上、下、右、左、前和后）中随机一个方向以不同百分比丢失部分形状的环境。结果如图 9 所示。在左侧，



PointNet 我们 地面真实

图 7. 比较部分分割结果。对于每个数据集，从左到右：PointNet、我们和地面真实。

显示了平均 IoU 与“保留比例”的关系。在右侧，对飞机模型的结果进行了可视化。

4.5 室内场景分割数据。我们在斯坦福大规模 3D 室内空间数据集（S3DIS）[Armeni 等人 2016]上评估我们的模型，用于语义场景分割任务。该数据集包含 6 个室内区域的 3D 扫描点云，总共 272 个房间。每个点属于 13 个语义类别中的一个——例如板、书架、椅子、天花板和梁——以及杂物物。我们遵循 Qi 等人[2017b]的相同设置，其中每个房间被分割成 $1\text{m} \times 1\text{m}$ 的块，每个点表示为一个 9 维向量（XYZ、RGB 和归一化空间坐标）。在训练过程中，每个块采样 4,096 个点，所有点用于测试。我们还使用相同的 6 折交叉验证在 6 个区域上进行，并报告了平均评估结果。

用于这项任务的模型与部分分割模型相似，不同之处在于为每个输入点生成语义对象类别的概率分布，并且这里不使用分类向量。我们将我们的模型与 PointNet [Qi 等人 2017b] 和 PointNet 基线进行比较，其中使用额外的点特征（局部点密度、局部曲率和法线）来构建手工特征，然后输入到 MLP 分类器中。我们进一步将我们的工作与 [Engelmann 等人 2017] 和 PointCNN [Li 等人 2018a] 进行比较。Engelmann 等人 [2017] 提出了扩大 3D 场景感受野的网络架构。他们的工作中提出了两种不同的方法：MS+CU 用于多尺度块特征与整合单元；G+RCU 用于网格块与循环

	MEAN	AREO	BAG	CAP	CAR	CHAIR	EAR PHONE	GUITAR	KNIFE	LAMP	LAPTOP	MOTOR	MUG	PISTOL	ROCKET	SKATE BOARD	TABLE
# SHAPES		2690	76	55	898	3758	69	787	392	1547	451	202	184	283	66	152	5271
POINTNET	83.7	83.4	78.7	82.5	74.9	89.6	73.0	91.5	85.9	80.8	95.3	65.2	93.0	81.2	57.9	72.8	80.6
POINTNET++	85.1	82.4	79.0	87.7	77.3	90.8	71.8	91.0	85.9	83.7	95.3	71.6	94.1	81.3	58.7	76.4	82.6
KD-NET	82.3	80.1	74.6	74.3	70.3	88.6	73.5	90.2	87.2	81.0	94.9	57.4	86.7	78.1	51.8	69.9	80.3
LOCALFEATURENET	84.3	86.1	73.0	54.9	77.4	88.8	55.0	90.6	86.5	75.2	96.1	57.3	91.7	83.1	53.9	72.5	83.8
PCNN	85.1	82.4	80.1	85.5	79.5	90.8	73.2	91.3	86.0	85.0	95.7	73.2	94.8	83.3	51.0	75.0	81.8
POINTCNN	86.1	84.1	86.45	86.0	80.8	90.6	79.7	92.3	88.4	85.3	96.1	77.2	95.3	84.2	64.2	80.0	83.0
Ours	85.2	84.0	83.4	86.7	77.8	90.6	74.7	91.2	87.5	82.8	95.7	66.3	94.9	81.1	63.5	74.5	82.6

Table 6. Part segmentation results on ShapeNet part dataset. Metric is mIoU(%) on points.

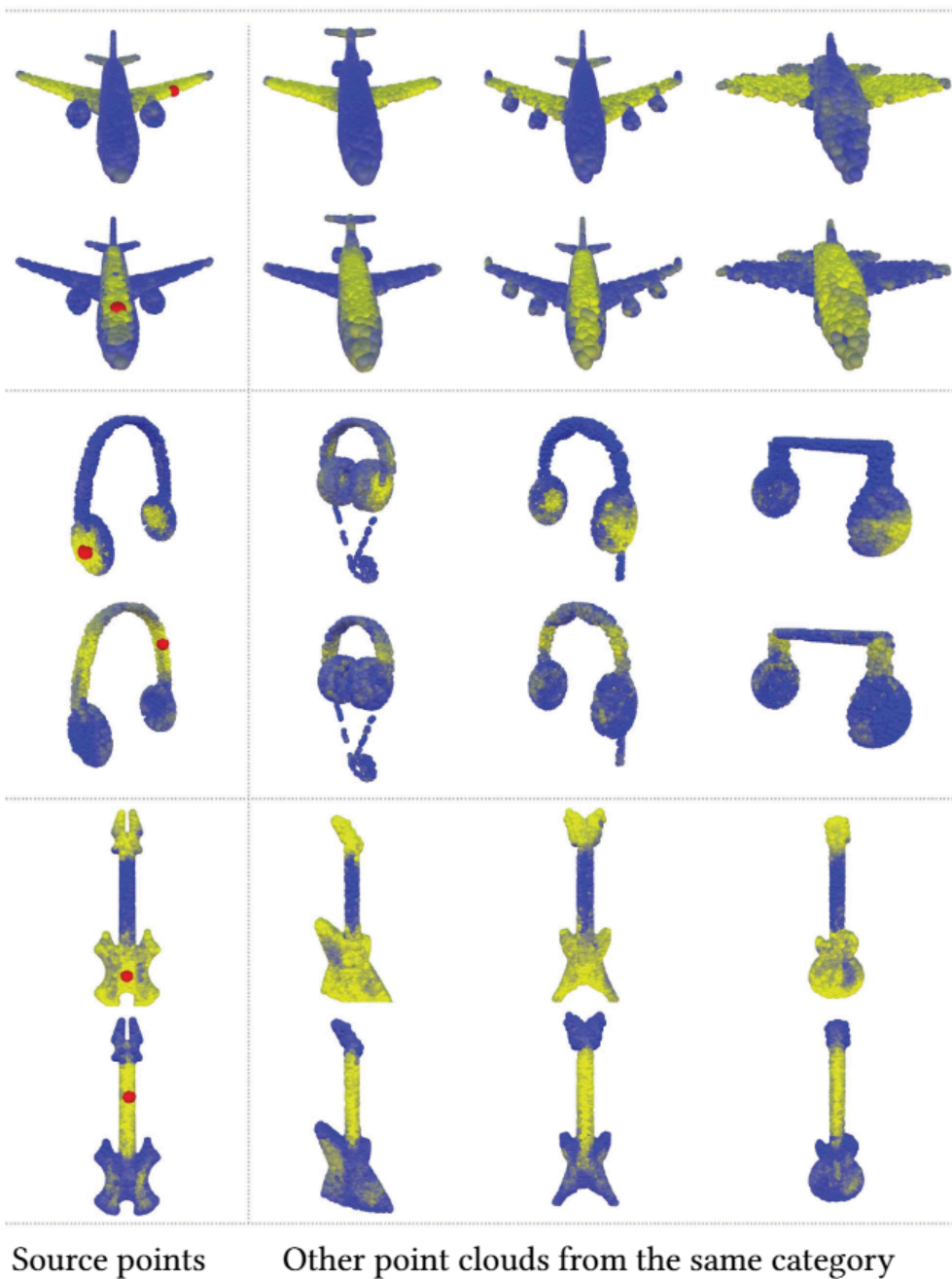


Fig. 8. Visualize the Euclidean distance (yellow: near, blue: far) between source points (red points in the left column) and multiple point clouds from the same category in the feature space after the third EdgeConv layer. Notice source points not only capture semantically similar structures in the point clouds that they belong to, but also capture semantically similar structures in other point clouds from the same category.

consolidation Units. We report evaluation results in Table 7, and visually compare the results of PointNet and our model in Figure 10.

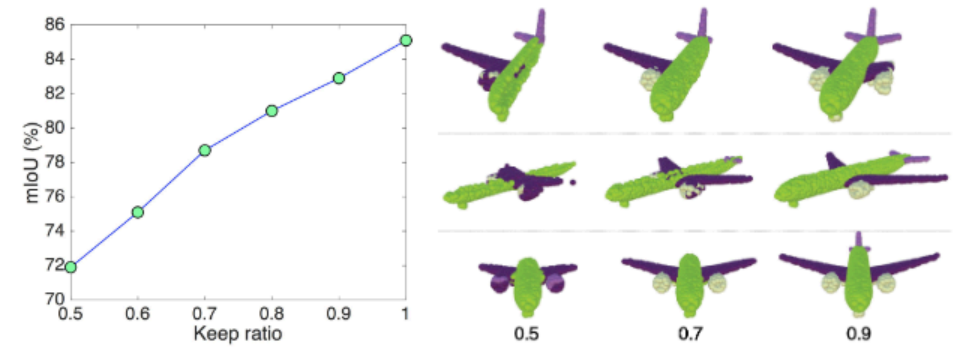


Fig. 9. **Left:** The mean IoU (%) improves when the ratio of kept points increases. Points are dropped from one of six sides (top, bottom, left, right, front and back) randomly during evaluation process. **Right:** Part segmentation results on partial data. Points on each row are dropped from the same side. The keep ratio is shown below the bottom row. Note that the segmentation results of turbines are improved when more points are included.

	MEAN IoU	OVERALL ACCURACY
POINTNET (BASELINE) [QI ET AL. 2017B]	20.1	53.2
POINTNET [QI ET AL. 2017B]	47.6	78.5
MS + CU(2) [ENGELMANN ET AL. 2017]	47.8	79.2
G + RCU [ENGELMANN ET AL. 2017]	49.7	81.1
POINTCNN [LI ET AL. 2018A]	65.39	-
Ours	56.1	84.1

Table 7. 3D semantic segmentation results on S3DIS. MS+CU for multi-scale block features with consolidation units; G+RCU for the grid-blocks with recurrent consolidation Units.

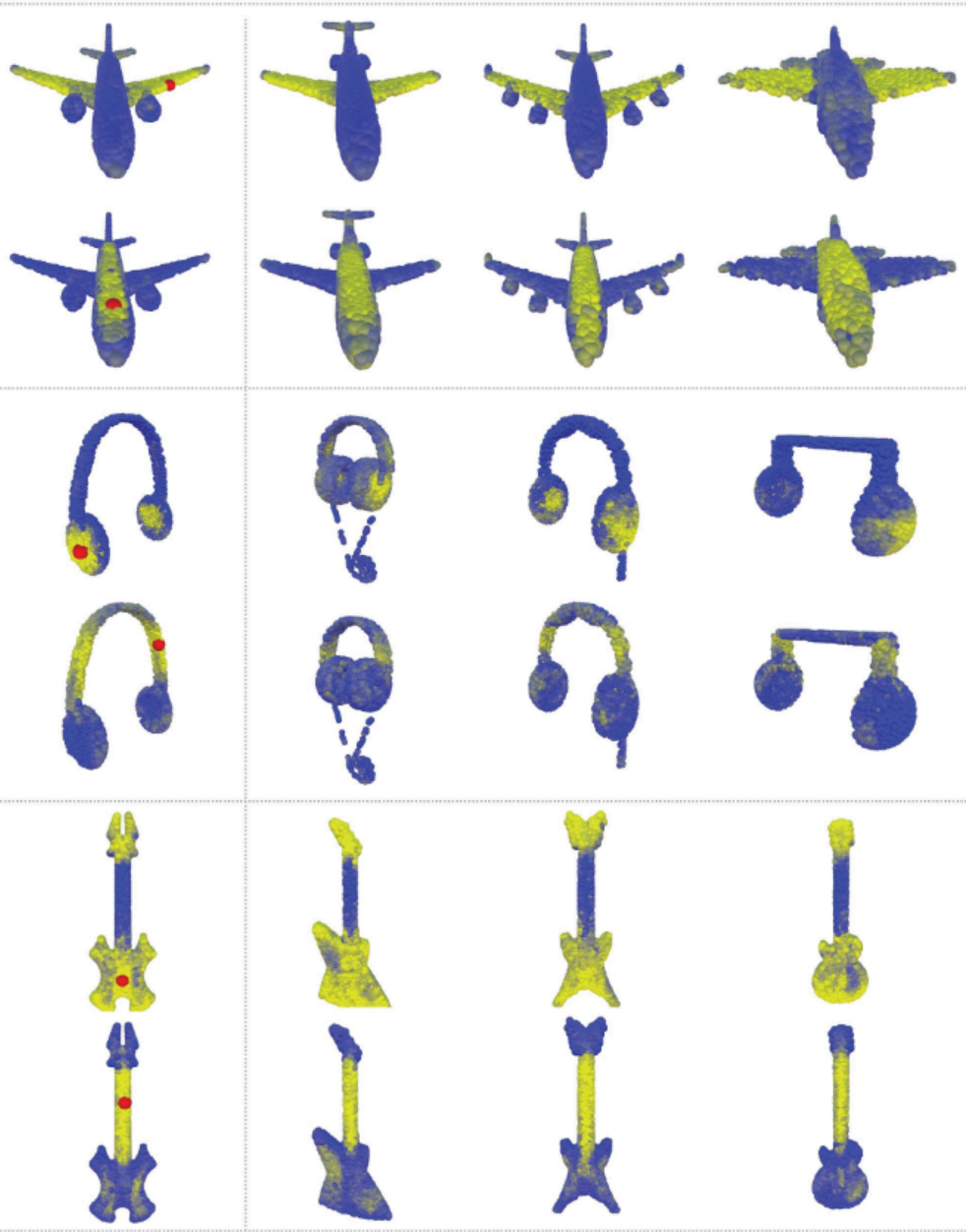
5 DISCUSSION

In this work we propose a new operator for learning on point cloud and show its performance on various tasks. Our model suggests that local geometric features are important to 3D recognition tasks, even after introducing machinery from deep learning.

While our architectures easily can be incorporated as-is into existing pipelines for point cloud-based graphics, learning, and vision, our experiments also indicate several avenues for future research and extension. Some details of our implementation could be revised and/or re-engineered to improve efficiency or scalability, e.g. incorporating fast data structures rather than computing pairwise

	均值	袋子	帽子	汽车	椅子	耳机	吉他	刀	灯	笔记本电脑	电机	杯子	手枪	火箭	滑板	桌子
手机	木板	# 形状	2690	76	55	898	3758	69	787	392	1547	451	202	184	283	66
PointNet 83.7 83.4 78.7 82.5 74.9 89.6 73.0 91.5 85.9 80.8 95.3 65.2 93.0 81.2 57.9 72.8 80.6																
PointNet++ 85.1 82.4 79.0 87.7 77.3 90.8 71.8 91.0 85.9 83.7 95.3 71.6 94.1 81.3 58.7 76.4 82.6																
Kd-Net 82.3 80.1 74.6 74.3 70.3 88.6 73.5 90.2 87.2 81.0 94.9 57.4 86.7 78.1 51.8 69.9 80.3																
LocalFeatureNet 84.3 86.1 73.0 54.9 77.4 88.8 55.0 90.6 86.5 75.2 96.1 57.3 91.7 83.1 53.9 72.5 83.8																
PCNN 85.1 82.4 80.1 85.5 79.5 90.8 73.2 91.3 86.0 85.0 95.7 73.2 94.8 83.3 51.0 75.0																
81.8 PointCNN 86.1 84.1 86.45 86.0 80.8 90.6 79.7 92.3 88.4 85.3 96.1 77.2 95.3 84.2 64.2 80.0 83.0																
Ours 85.2 84.0 83.4 86.7 77.8 90.6 74.7 91.2 87.5 82.8 95.7 66.3 94.9 81.1 63.5 74.5 82.6																

表 6. ShapeNet 部件数据集上的部分分割结果。指标是点上的 mIoU(%)。



源点 同类其他点云

图 8. 在经过第三次 EdgeConv 层后的特征空间中，可视化源点（左列中的红点）与同一类别中多个点云之间的欧几里得距离（黄色：近，蓝色：远）。注意源点不仅捕获了它们所属点云中的语义相似结构，还捕获了同一类别其他点云中的语义相似结构。

聚合单元。我们在表 7 中报告了评估结果，并在图 10 中直观比较了 PointNet 和我们模型的结果。

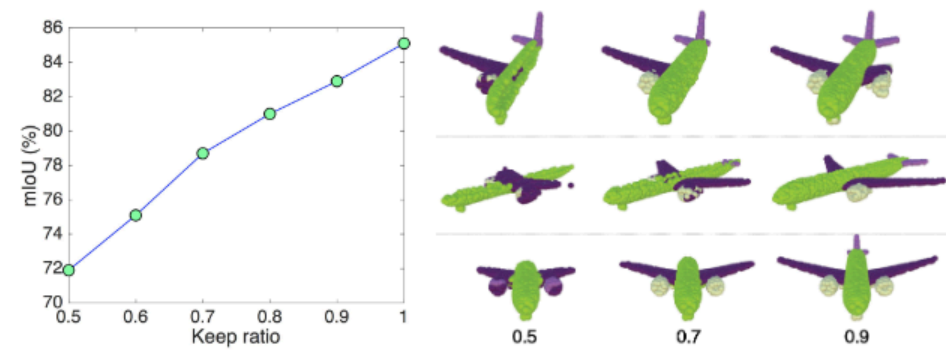


图 9. 左：当保留点的比例增加时，平均 IoU (%) 会提高。在评估过程中，点会从六个方向（上、下、左、右、前和后）中随机丢弃。右：部分数据的部分分割结果。每行上的点是从同一方向丢弃的。保留比例显示在底行下方。请注意，当包含更多点时，风机的分割结果会得到改善。

平均总体	
IoU 精度	PointNet (基线) [Qi 等人 2017b] 20.1 53.2 PointNet [Qi 等人 2017b] 47.6 78.5 MS + CU(2) [Engelmann 等人 2017] 47.8 79.2 G + RCU [Engelmann 等人 2017] 49.7 81.1 PointCNN [Li 等人 2018a] 65.39 -
我们	56.1 84.1

表 7. S3DIS 上的 3D 语义分割结果。MS+CU 为多尺度块特征与整合单元；G+RCU 为带循环整合单元的网格块。

5 讨论

在这项工作中，我们提出了一种新的点云学习算子，并展示了其在各种任务上的性能。我们的模型表明，局部几何特征对 3D 识别任务仍然重要，即使在引入深度学习技术之后也是如此。虽然我们的架构可以轻松地直接集成到现有的基于点云的图形、学习和视觉流程中，但我们的实验也指出了未来研究和扩展的几个方向。我们实现的某些细节可以修订和/或重新设计以提高效率或可扩展性，例如采用快速数据结构而不是计算成对

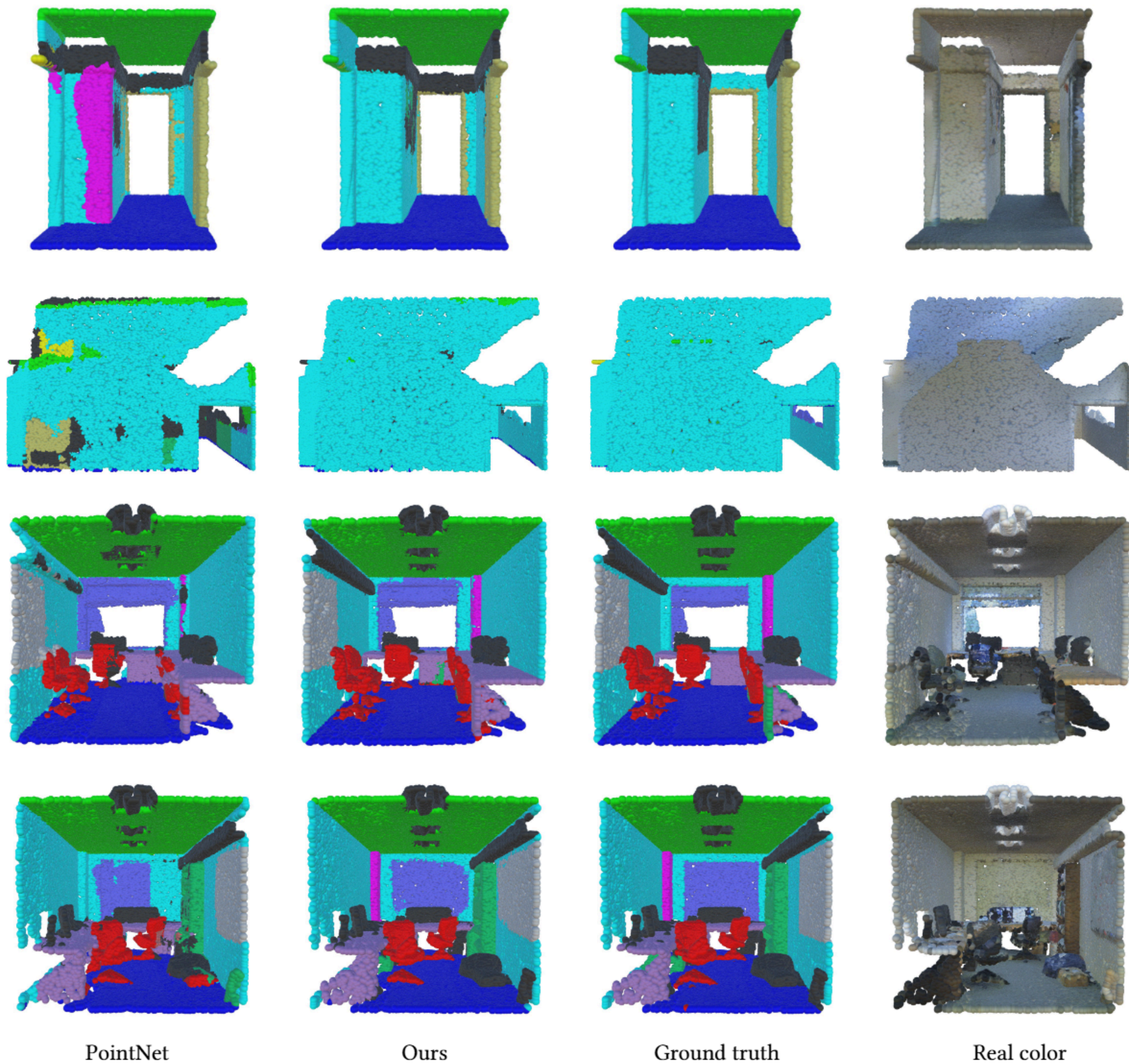


Fig. 10. Semantic segmentation results. From left to right: PointNet, ours, ground truth and point cloud with original color. Notice our model outputs smoother segmentation results, for example, wall (cyan) in top two rows, chairs (red) and columns (magenta) in bottom two rows.

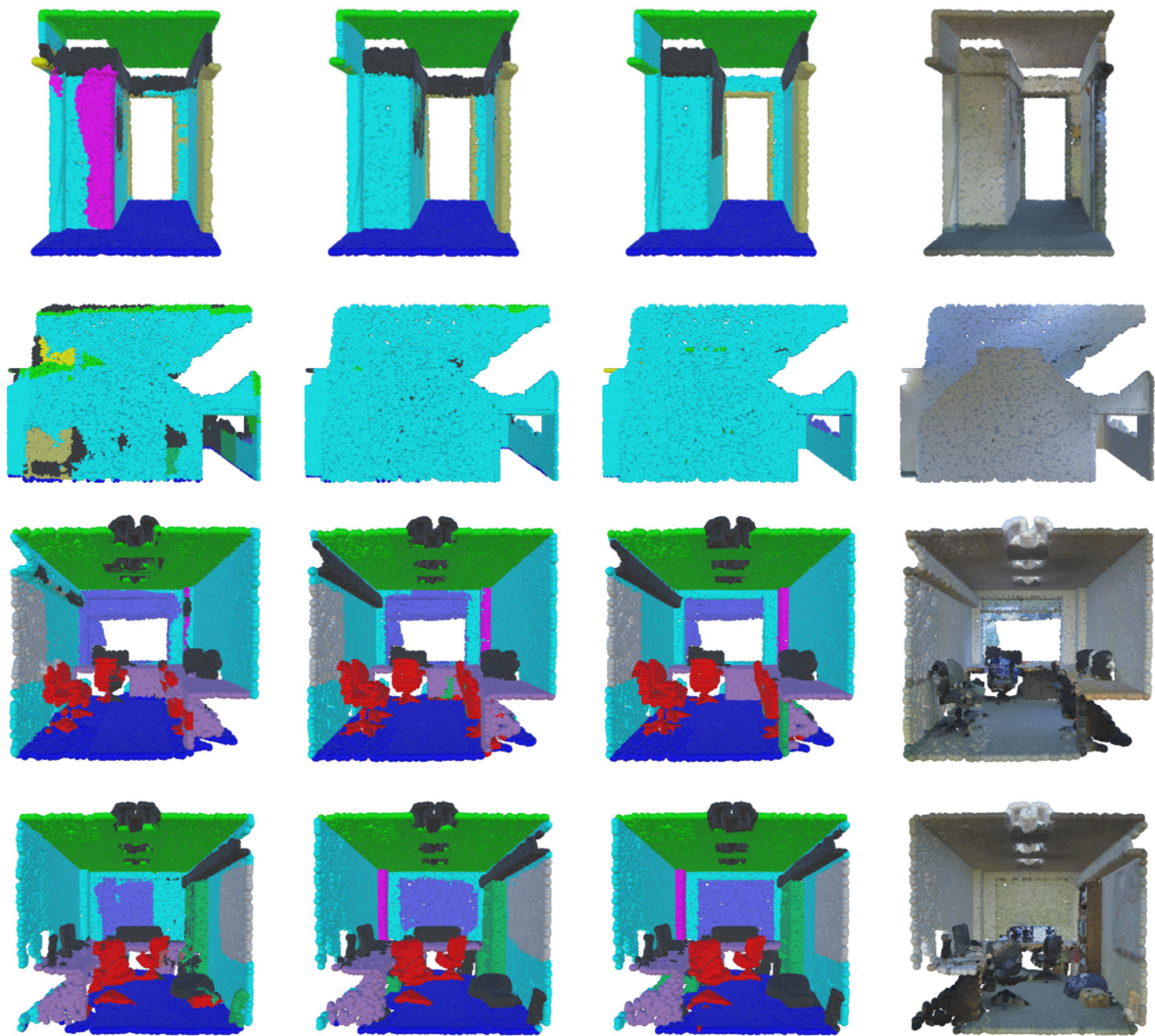
distances to evaluate k -nearest neighbors queries. We also could consider higher-order relationships between larger tuples of points, rather than considering them pairwise. Another possible extension is to design a non-shared transformer network that works on each local patch differently, adding flexibility to our model.

Our experiments suggest that intrinsic features can be equally valuable if not more valuable than point coordinates; developing a practical and theoretically-justified framework for balancing intrinsic and extrinsic considerations in a learning pipeline will require insight from theory and practice in geometry processing. Given this, we will consider applications of our techniques to more abstract point clouds coming from applications like document retrieval and image processing rather than 3D geometry; beyond broadening

the applicability of our technique, these experiments will provide insight into the role of geometry in abstract data processing.

ACKNOWLEDGMENTS

The authors acknowledge the generous support of Army Research Office grant W911NF-12-R-0011, of Air Force Office of Scientific Research award FA9550-19-1-0319, of National Science Foundation grant IIS-1838071, of ERC Consolidator grant No. 724228 (LEMAN), from an Amazon Research Award, from the MIT-IBM Watson AI Laboratory, from the Toyota-CSAIL Joint Research Center, from the Skoltech-MIT Next Generation Program, and from Google Faculty



PointNet 我们的真实颜色

图 10. 语义分割结果。从左到右：PointNet、我们的模型、真实标签和原始颜色的点云。注意我们的模型输出更平滑的分割结果，例如，顶部两行的墙壁（青色）、底部两行的椅子（红色）和柱子（品红色）。

用于评估 k 近邻查询的距离。我们还可以考虑点之间更大元组的更高阶关系，而不仅仅是成对考虑。另一个可能的扩展是设计一个非共享的 Transformer 网络，该网络以不同的方式作用于每个局部区域，为我们的模型增加灵活性。

我们的实验表明，内在特征与点坐标一样有价值，甚至更有价值；开发一个实用的、有理论依据的框架，以平衡学习流程中内在和外在的考虑，需要几何处理理论和实践的洞察。鉴于这一点

我们将考虑将我们的技术应用于来自文档检索和图像处理等应用中的更抽象的点云，而不是 3D 几何；除了拓宽我们技术的适用范围之外，这些实验还将提供对几何在抽象数据处理中作用的见解。

致谢

作者感谢陆军研究办公室的慷慨支持，包括赠款 W911NF-12-R-0011，空军科学研究办公室的奖项 FA9550-19-1-0319，国家自然科学基金会的赠款 IIS-1838071，ERC 整合基金的赠款 No. 724228 (LEMAN)，来自亚马逊研究奖，来自麻省理工学院-IBM 沃森人工智能实验室，来自丰田-CSAIL 联合研究中心，来自斯克勒茨基-麻省理工学院下一代计划，以及来自谷歌教职员工

Research Award. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of these organizations.

REFERENCES

- Iro Armeni, Ozan Sener, Amir R. Zamir, Helen Jiang, Ioannis Brilakis, Martin Fischer, and Silvio Savarese. 2016. 3D Semantic Parsing of Large-Scale Indoor Spaces. In *Proc. CVPR*.
- Matan Atzmon, Haggai Maron, and Yaron Lipman. 2018. Point Convolutional Neural Networks by Extension Operators. *ACM Trans. Graph.* 37, 4, Article 71 (July 2018), 12 pages. <https://doi.org/10.1145/3197517.3201301>
- Mathieu Aubry, Ulrich Schlickewei, and Daniel Cremers. 2011. The Wave Kernel Signature: A Quantum Mechanical Approach to Shape Analysis. In *Proc. ICCV Workshops*.
- Serge Belongie, Jitendra Malik, and Jan Puzicha. 2001. Shape Context: A New Descriptor for Shape Matching and Object Recognition. In *Proc. NIPS*.
- Silvia Biasotti, Andrea Cerri, A Bronstein, and M Bronstein. 2016. Recent Trends, Applications, and Perspectives in 3D Shape Similarity assessment. *Computer Graphics Forum* 35, 6 (2016), 87–119.
- Davide Boscaini, Jonathan Masci, Emanuele Rodolà, and Michael Bronstein. 2016. Learning Shape Correspondence with Anisotropic Convolutional Neural Networks. In *Proc. NIPS*.
- Andrew Brock, Theodore Lim, James Millar Ritchie, and Nicholas J. Weston. 2016. Generative and Discriminative Voxel Modeling with Convolutional Neural Networks. In *Proc. NIPS*.
- Michael M Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, and Pierre Vandergheynst. 2017. Geometric Deep Learning: Going beyond Euclidean Data. *IEEE Signal Processing Magazine* 34, 4 (2017), 18–42.
- Michael M Bronstein and Iasonas Kokkinos. 2010. Scale-invariant Heat Kernel Signatures for Non-rigid Shape Recognition. In *Proc. CVPR*.
- Joan Bruna, Wojciech Zaremba, Arthur Szlam, and Yann LeCun. 2013. Spectral Networks and Locally Connected Networks on Graphs. *arXiv:1312.6203* (2013).
- Angel X Chang, Thomas Funkhouser, Leonidas Guibas, Pat Hanrahan, Qixing Huang, Zimo Li, Silvio Savarese, Manolis Savva, Shuran Song, Hao Su, et al. 2015. Shapenet: An Information-rich 3D Model Repository. *arXiv:1512.03012* (2015).
- Michaël Defferrard, Xavier Bresson, and Pierre Vandergheynst. 2016. Convolutional Neural Networks on Graphs with Fast Localized Spectral Filtering. In *Proc. NIPS*.
- Francis Engelmann, Theodora Kontogianni, Alexander Hermans, and Bastian Leibe. 2017. Exploring Spatial Context for 3D Semantic Segmentation of Point Clouds. In *Proc. CVPR*.
- Danielle Ezuz, Justin Solomon, Vladimir G Kim, and Mirela Ben-Chen. 2017. GWCNN: A Metric Alignment Layer for Deep Shape Analysis. *Computer Graphics Forum* 36, 5 (2017), 49–57.
- Haoqiang Fan, Hao Su, and Leonidas J Guibas. 2017. A Point Set Generation Network for 3D Object Reconstruction from a Single Image.. In *Proc. CVPR*.
- Matthias Fey, Jan Eric Lenssen, Frank Weichert, and Heinrich Müller. 2018. SplineCNN: Fast Geometric Deep Learning with Continuous B-Spline Kernels. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, and George E Dahl. 2017. Neural Message Passing for Quantum Chemistry. *arXiv:1704.01212* (2017).
- Aleksey Golovinskiy, Vladimir G. Kim, and Thomas Funkhouser. 2009. Shape-based Recognition of 3D Point Clouds in Urban Environments. In *Proc. ICCV*.
- Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative Adversarial Nets. In *Proc. NIPS*.
- Paul Guerrero, Yanir Kleiman, Maks Ovsjanikov, and Niloy J. Mitra. 2018. PCPNet: Learning Local Shape Properties from Raw Point Clouds. *Computer Graphics Forum* 37, 2 (2018), 75–85. <https://doi.org/10.1111/cgf.13343>
- Yulan Guo, Mohammed Bannamoun, Ferdous Sohel, Min Lu, and Jianwei Wan. 2014. 3D Object Recognition in Cluttered Scenes with Local Surface Features: a Survey. *Trans. PAMI* 36, 11 (2014), 2270–2287.
- Oshri Halimi, Or Litany, Emanuele Rodolà, Alex Bronstein, and Ron Kimmel. 2018. Self-supervised Learning of Dense Shape Correspondence. *arXiv:1812.02415* (2018).
- M. Henaff, J. Bruna, and Y. LeCun. 2015. Deep Convolutional Networks on Graph-structured Data. *arXiv:1506.05163* (2015).
- Andrew E. Johnson and Martial Hebert. 1999. Using Spin Images for Efficient Object Recognition in Cluttered 3D Scenes. *Trans. PAMI* 21, 5 (1999), 433–449.
- Diederik P Kingma and Max Welling. 2013. Auto-encoding Variational Bayes. *arXiv:1312.6114* (2013).
- Thomas N Kipf and Max Welling. 2017. Semi-supervised Classification with Graph Convolutional Networks. (2017).
- Roman Klokov and Victor Lempitsky. 2017. Escape from Cells: Deep Kd-Networks for The Recognition of 3D Point Cloud Models. (2017).
- Ilya Kostrikov, Zhongshi Jiang, Daniele Panozzo, Denis Zorin, and Joan Bruna. 2017. Surface Networks. In *Proc. CVPR*.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. Imagenet Classification with Deep Convolutional Neural Networks. In *Proc. NIPS*.
- Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard, and Lawrence D Jackel. 1989. Backpropagation Applied to Hand-written ZIP Code Recognition. *Neural computation* 1, 4 (1989), 541–551.
- Ron Levie, Federico Monti, Xavier Bresson, and Michael M Bronstein. 2017. CayleyNets: Graph Convolutional Neural Networks with Complex Rational Spectral Filters. *arXiv:1705.07664* (2017).
- Chun-Liang Li, Manzil Zaheer, Yang Zhang, Barnabas Poczos, and Ruslan Salakhutdinov. 2018b. Point Cloud GAN. *arXiv:1810.05795* (2018).
- Yangyan Li, Rui Bu, Mingchao Sun, Wei Wu, Xinhan Di, and Baoquan Chen. 2018a. PointCNN: Convolution On X-Transformed Points. In *Advances in Neural Information Processing Systems 31*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.). Curran Associates, Inc., 820–830. <http://papers.nips.cc/paper/7362-pointcnn-convolution-on-x-transformed-points.pdf>
- Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2016. Gated graph Sequence Neural Networks. In *Proc. ICLR*.
- Ming Liang, Bin Yang, Shenlong Wang, and Raquel Urtasun. 2018. Deep Continuous Fusion for Multi-Sensor 3D Object Detection. In *The European Conference on Computer Vision (ECCV)*.
- Haibin Ling and David W Jacobs. 2007. Shape Classification using the Inner-distance. *Trans. PAMI* 29, 2 (2007), 286–299.
- Or Litany, Alex Bronstein, Michael Bronstein, and Ameesh Makadia. 2017a. Deformable Shape Completion with Graph Convolutional Autoencoders. *arXiv:1712.00268* (2017).
- Or Litany, Tal Remez, Emanuele Rodolà, Alex M Bronstein, and Michael M Bronstein. 2017b. Deep Functional Maps: Structured Prediction for Dense Shape Correspondence. In *Proc. ICCV*.
- I. Loshchilov and F. Hutter. 2017. SGDR: Stochastic Gradient Descent with Warm Restarts. In *International Conference on Learning Representations (ICLR) 2017 Conference Track*.
- Min Lu, Yulan Guo, Jun Zhang, Yanxin Ma, and Yinjie Lei. 2014. Recognizing Objects in 3D Point Clouds with Multi-scale Local Features. *Sensors* 14, 12 (2014), 24156–24173.
- Siddharth Manay, Daniel Cremers, Byung-Woo Hong, Anthony J Yezzi, and Stefano Soatto. 2006. Integral Invariants for Shape Matching. *Trans. PAMI* 28, 10 (2006), 1602–1618.
- Haggai Maron, Meirav Galun, Noam Aigerman, Miri Trope, Nadav Dym, Ersin Yumer, Vladimir G Kim, and Yaron Lipman. 2017. Convolutional Neural Networks on Surfaces via Seamless Toric Covers. In *Proc. SIGGRAPH*.
- Jonathan Masci, Davide Boscaini, Michael Bronstein, and Pierre Vandergheynst. 2015. Geodesic Convolutional Neural Networks on Riemannian Manifolds. In *Proc. 3dRRR*.
- Daniel Maturana and Sebastian Scherer. 2015. Voxnet: A 3D Convolutional Neural Network for Real-time Object Recognition. In *Proc. IROS*.
- Federico Monti, Davide Boscaini, Jonathan Masci, Emanuele Rodolà, Jan Svoboda, and Michael M Bronstein. 2017a. Geometric Deep Learning on Graphs and Manifolds using Mixture Model CNNs. In *Proc. CVPR*.
- F. Monti, M. M. Bronstein, and X. Bresson. 2017b. Geometric Matrix Completion with Recurrent Multi-graph Neural Networks. In *Proc. NIPS*.
- Federico Monti, Karl Otness, and Michael M Bronstein. 2018. MotifNet: A Motif-based Graph Convolutional Network for Directed Graphs. *arXiv:1802.01572* (2018).
- Maks Ovsjanikov, Mirela Ben-Chen, Justin Solomon, Adrian Butscher, and Leonidas J Guibas. 2012. Functional Maps: A Flexible Representation of Maps between Shapes. *TOG* 31, 4 (2012), 30.
- Charles R Qi, Wei Liu, Chenxia Wu, Hao Su, and Leonidas J Guibas. 2017a. Frustum PointNets for 3D Object Detection from RGB-D Data. *arXiv:1711.08488* (2017).
- Charles R. Qi, Hao Su, Kaichun Mo, and Leonidas J. Guibas. 2017b. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *Proc. CVPR*.
- Charles R Qi, Hao Su, Matthias Nießner, Angela Dai, Mengyuan Yan, and Leonidas J Guibas. 2016. Volumetric and Multi-view CNNs for Object Classification on 3D Data. In *Proc. CVPR*.
- Charles R. Qi, Li Yi, Hao Su, and Leonidas J. Guibas. 2017c. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In *Proc. NIPS*.
- Anurag Ranjan, Timo Bolkart, Soubhik Sanyal, and Michael J Black. 2018. Generating 3D faces using Convolutional Mesh Autoencoders. *arXiv:1807.10267* (2018).
- Raif M Rustamov. 2007. Laplace-Beltrami Eigenfunctions for Deformation Invariant Shape Representation. In *Proc. SGP*.
- Radu Bogdan Rusu, Nico Blodow, and Michael Beetz. 2009. Fast Point Feature Histograms (FPFH) for 3D Registration. In *Proc. ICRA*.
- Radu Bogdan Rusu, Nico Blodow, Zoltan Csaba Marton, and Michael Beetz. 2008a. Aligning Point Cloud Views using Persistent Feature Histograms. In *Proc. IROS*.
- Radu Bogdan Rusu, Zoltan Csaba Marton, Nico Blodow, Mihai Dolha, and Michael Beetz. 2008b. Towards 3D Point Cloud Based Object Maps for Household Environments. *Robotics and Autonomous Systems Journal* 56, 11 (30 November 2008), 927–941.
- Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, and Gabriele Monfardini. 2009. The Graph Neural Network Model. *IEEE Tran. Neural Networks* 20, 1 (2009), 61–80.

1:12 · 王越, 孙永斌, 刘子伟, Sanjay E. Sarma, Michael M. Bronstein, 和 Justin M. Solomon

研究奖。本材料中表达的任何意见、发现、结论或建议均为作者所有，不一定反映这些组织的观点。

参考文献

Iro Armeni, Ozan Sener, Amir R. Zamir, Helen Jiang, Ioannis Brilakis, Martin Fischer, 和 Silvio Savarese. 2016. 大规模室内空间的 3D 语义解析. CVPR 会议论文集.

Matan Atzmon, Haggai Maron, 和 Yaron Lipman. 2018. 通过扩展算子实现的点卷积神经网络. ACM Transactions on Graphics, 37, 4, 文章 71 (2018 年 7 月), 12 页. <https://doi.org/10.1145/3197517.3201301> Mathieu Aubry, Ulrich Schlickewei, 和 Daniel Cremers. 2011. 波核签名：一种用于形状分析的量子力学方法. ICCV 研讨会会议论文集.

Serge Belongie, Jitendra Malik, 和 Jan Puzicha. 2001. 形状上下文：一种用于形状匹配和物体识别的新描述符. NIPS 会议论文集. Silvia Biasotti, Andrea Cerri, A Bronstein, 和 M Bronstein. 2016. 3D 形状相似性评估的最新趋势、应用和前景. Computer Graphics Forum, 35, 6 (2016 年), 87–119.

Davide Boscaini, Jonathan Masci, Emanuele Rodolà, 和 Michael Bronstein. 2016.

使用各向异性卷积神经网络学习形状对应关系. 发表于 NIPS 会议论文集.

Andrew Brock, Theodore Lim, James Millar Ritchie, 和 Nicholas J. Weston. 2016. 基于卷积神经网络的生成与判别体素建模. 发表于 NIPS 会议论文集.

Michael M Bronstein, Joan Bruna, Yann LeCun, Arthur Szlam, 和 Pierre Vandergheynst. 2017 年. 几何深度学习：超越欧几里得数据. IEEE 信号处理杂志第 34 卷第 4 期（2017 年），第 18-42 页.

Michael M Bronstein 和我. 2010 年。《尺度不变的热核签名用于非刚性形状识别》。在 CVPR 会议论文集中。Joan Bruna、Wojciech Zaremba、Arthtur Szlam 和 Yann LeCun。2013 年。《图上的谱网络和局部连接网络》。arXiv:1312.6203 (2013)。Angel X Chang、Thomas Funkhouser、Leonidas Guibas、Pat Hanrahan、Qixing Huang、

Zimo Li、Silvio Savarese、Manolis Savva、Shuran Song、Hao Su 等。2015 年。《ShapeNet：一个信息丰富的 3D 模型库》。arXiv:1512.03012 (2015)。

Michaël Defferrard、Xavier Bresson 和 Pierre Vandergheynst. 2016 年。《图上的卷积神经网络与快速局部化谱过滤》。在 NIPS 会议论文集中。

Francis Engelmann、Theodora Kontogianni、Alexander Hermans 和 Bastian Leibe。2017. 探索点云三维语义分割的空间上下文. CVPR 会议论文.

Danielle Ezuz, Justin Solomon, Vladimir G Kim, 和 Mirela Ben-Chen. 2017. GWCNN: 用于深度形状分析的度量对齐层. Computer Graphics Forum 36, 5 (2017), 49–57. Haoqiang Fan, Hao Su, 和 Leonidas J Guibas. 2017. 用于从单张图像进行三维物体重建的点集生成网络. CVPR 会议论文. Matthias Fey, Jan Eric Lenssen, Frank Weichert, 和 Heinrich Müller. 2018. SplineCNN: 基于连续 B 样条核的快速几何深度学习. CVPR 会议论文. Justin Gilmer, Samuel S Schoenholz, Patrick F Riley, Oriol Vinyals, 和 George E Dahl.

2017. 用于量子化学的神经消息传递. arXiv:1704.01212 (2017).

Aleksey Golovinskiy, Vladimir G. Kim, 和 Thomas Funkhouser. 2009. 城市环境中的三维点云形状识别. ICCV 会议论文. Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, 和 Yoshua Bengio. 2014. 生成对抗网络. NIPS 会议论文.

保罗·格雷罗、亚尼·克莱曼、马克斯·奥夫申尼科夫和尼尔奥伊·米特拉。2018 年。《PCPNet：从原始点云中学习局部形状属性》。计算机图形论坛 37, 2 (2018), 75–85。 <https://doi.org/10.1111/cgf.13343>

郭玉兰、穆罕默德·本纳穆恩、费尔杜斯·索赫尔、陆敏和万建伟。2014 年。《杂乱场景中的 3D 物体识别：基于局部表面特征的综述》。PAMI 36, 11 (2014), 2270–2287。

奥希里·哈利米、奥·利坦尼、埃马纽埃尔·罗多拉、亚历克斯·布罗斯坦和罗恩·基梅尔。2018 年。自监督学习密集形状对应。arXiv:1812.02415 (2018)。

M. Henaff、J. Bruna 和 Y. LeCun。2015。图结构数据上的深度卷积网络。arXiv:1506.05163 (2015)。Andrew E. Johnson 和 Martial Hebert. 1999. 使用自旋图像在杂乱 3D 场景中进行高效物体识别. PAMI 汇刊 21, 5 (1999), 433–449。

Diederik P Kingma 和 Max Welling。2013。自编码变分贝叶斯。

arXiv:1312.6114 (2013)。

托马斯·N·基普夫和马克斯·韦林。2017 年。基于图卷积网络的半监督分类。(2017 年)。

罗马·克洛科夫和维克多·伦皮茨基。2017 年。逃离单元：用于 3D 点云模型识别的深度 Kd 网络。(2017 年)。

伊利亚·科斯特罗夫、钟世江、达尼尔·帕努佐、丹尼斯·佐林和乔恩·布劳纳。2017 年。

表面网络。在 CVPR 会议论文集中。

ACM 图形学汇刊，第 1 卷，第 1 期，文章 1。出版日期：2019 年 1 月。

亚历克斯·克里希纳夫斯基、伊利亚·苏茨凯弗和杰弗里·E·希顿。2012 年。使用深度卷积神经网络进行 ImageNet 分类。在 NIPS 会议论文集中。扬恩·勒库恩、伯纳德·博瑟、约翰·S·登克、唐尼·亨德森、理查德·E·霍华德、韦恩·哈伯德和劳伦斯·D·杰克尔。1989 年。反向传播应用于手写 ZIP 码识别。神经网络计算 1, 4 (1989 年), 541–551。罗恩·列维、费德里科·蒙蒂、夏尔·布雷斯顿和迈克尔·M·布罗斯坦。2017 年。凯莱网络：具有复数有理谱滤波器的图卷积神经网络。

arXiv:1705.07664 (2017).

陈春亮, 曼齐尔·扎黑尔, 张阳, 巴纳巴斯·波科佐斯, 和拉斯拉纳·萨拉赫乌丁诺夫。2018b. 点云 GAN。arXiv:1810.05795 (2018).

李阳岩, 卜瑞, 孙明超, 吴伟, 狄新汉, 和陈宝权。2018a.

PointCNN: 在 X 变换点上进行的卷积。发表于《神经信息处理系统进展》第 31 卷, S. Bengio、H. Wallach、H. Larochelle、K. Grauman、N. Cesa-Bianchi 和 R. Garnett（编）。Curran Associates, Inc., 820–830。http://papers.nips.cc/paper/7362-pointcnn-convolution-on-x-transformed-points.pdf Yujia Li, Daniel Tarlow, Marc Brockschmidt, and Richard Zemel. 2016. Gated graph Sequence Neural Networks.发表于《国际机器学习会议论文集》。

Ming Liang, Bin Yang, Shenlong Wang, and Raquel Urtasun. 2018. Deep Continuous Fusion for Multi-Sensor 3D Object Detection.发表于《欧洲计算机视觉会议》(ECCV)。

Haibin Ling and David W Jacobs. 2007. Shape Classification using the Inner-distance。

《模式、图像与图像处理》第 29 卷，第 2 期（2007 年），286–299。

Or Litany, Alex Bronstein, Michael Bronstein 和 Ameesh Makadia. 2017a。《基于图卷积自动编码器的可变形形状补全》。arXiv:1712.00268（2017）。

Or Litany, Tal Remez, Emanuele Rodolà, Alex M Bronstein 和 Michael M Bronstein。

2017b。《深度功能图：用于密集形状对应的结构化预测》。在 ICCV 会议论文集。

I. Loshchilov 和 F. Hutter。2017。《SGDR：带预热随机梯度下降》。重启。在国际学习表示会议（ICLR）2017 会议轨道。

Min Lu, Yulan Guo, Jun Zhang, Yanxin Ma, 和 Yinjie Lei. 2014. 在 3D 点云中识别物体，使用多尺度局部特征。传感器 14, 12 (2014), 24156–24173. Siddharth Manay, Daniel Cremers, Byung-Woo Hong, Anthony J Yezzi, 和 Stefano Soatto. 2006. 形状匹配的积分不变量。PAMI 传输 28, 10 (2006), 1602–1618. Haggai Maron, Meirav Galun, Noam Aigerman, Miri Trope, Nadav Dym, Ersin Yumer, Vladimir G Kim, 和 Yaron Lipman. 2017. 表面上的卷积神经网络，通过无缝缝里克覆盖。在 SIGGRAPH 会议记录。

Jonathan Masci, Davide Boscaini, Michael Bronstein, 和 Pierre Vandergheynst. 2015.

黎曼流形上的测地卷积神经网络。在 3dRR 会议记录。

丹尼尔·马图拉纳和塞巴斯蒂安·舍尔尔. 2015. Voxnet: 一种用于实时目标识别的 3D 卷积神经网络. 在 IROS 会议论文集中. 费德里科·蒙蒂, 达维德·博斯卡尼, 乔纳森·马斯西, 埃马纽埃尔·罗多拉, 扬·斯沃博达, 和迈克尔·M·布罗斯坦. 2017a. 基于混合模型 CNN 的图和流形几何深度学习. 在 CVPR 会议论文集中. F.蒙蒂, M.M.布罗斯坦, 和 X.布雷松. 2017b. 基于循环多图神经网络的几何矩阵补全. 在 NIPS 会议论文集中. 费德里科·蒙蒂, 卡尔·奥滕斯, 和迈克尔·M·布罗斯坦. 2018. MotifNet: 一种基于模式的图卷积网络用于有向图. arXiv:1802.01572 (2018). 马克斯·奥夫申尼科夫, 米雷拉·本-切恩, 贾斯汀·索洛蒙, 亚当·布特舍尔, 和利奥尼达斯·吉巴斯. 2012. 功能映射: 形状之间映射的灵活表示.

TOG 31, 4 (2012), 30.

查尔斯·R·齐, 魏柳, 吴晨霞, 苏浩, 以及利奥尼达斯·J·古伊巴斯。2017a。《基于 RGB-D 数据的视锥点云网络进行 3D 目标检测》。arXiv:1711.08488 (2017)。查尔斯·R.齐, 苏浩, 莫凯春, 以及利奥尼达斯·J.古伊巴斯。2017b。《点云深度学习：用于 3D 分类和分割的点集》。在 CVPR 会议论文集。查尔斯·R Qi, 苏浩, 马蒂亚斯·尼施纳, 戴安琪拉, 闫梦媛, 以及利奥尼达斯·J Guibas。2016。《体素和多视图卷积神经网络用于 3D 数据的对象分类》。在 CVPR 会议论文集。

查尔斯·R.齐, 李毅, 苏浩, 以及利奥尼达斯·J.古伊巴斯。2017c。《点云++：度量空间中点集的深度层次特征学习》。在 NIPS 会议论文集。阿努拉格·兰贾恩, 蒂莫·博尔卡特, 桑吉克·萨尼亚尔, 以及迈克尔·J·布莱克。2018。《使用卷积网格自动编码器生成 3D 人脸》。arXiv:1807.10267 (2018)。拉伊夫·M·鲁斯坦诺夫。2007。《拉普拉斯-贝尔特拉米特征函数用于变形不变形状表示》。在 SGP 会议论文集。拉杜·博格丹·鲁苏, 尼科·布洛多, 以及迈克尔·贝茨。2009。《快速点特征直方图（FPFH）用于 3D 配准》。在 ICRA 会议论文集。

拉杜·博格丹·鲁苏, 尼科·布洛多, 马特亚什·查巴·马顿, 以及迈克尔·贝茨。2008a。

使用持久特征直方图对齐点云视图。发表于 IROS 会议论文集。

Radu Bogdan Rusu, Zoltan Csaba Marton, Nico Blodow, Mihai Dolha, 和 Michael Beetz。2008b. 基于三维点云的家庭环境物体地图研究。

《机器人与自动化系统》杂志第 56 卷，第 11 期（2008 年 11 月 30 日），第 927-941 页。Franco Scarselli, Marco Gori, Ah Chung Tsoi, Markus Hagenbuchner, 和 Gabriele Monfardini. 2009. 图神经网络模型. IEEE 神经网络汇刊第 20 卷，第 1 期（2009 年），第 61-80 页。

- Syed Afaq Ali Shah, Mohammed Bennamoun, Farid Boussaid, and Amar A El-Sallam. 2013. 3D-Div: A novel Local Surface Descriptor for Feature Matching and Pairwise Range Image Registration. In *Proc. ICIP*.
- Yiru Shen, Chen Feng, Yaoqing Yang, and Dong Tian. 2017. Neighbors Do Help: Deeply Exploiting Local Structures of Point Clouds. *arXiv:1712.06760* (2017).
- David I Shuman, Sunil K Narang, Pascal Frossard, Antonio Ortega, and Pierre Vandergheynst. 2013. The Emerging Field of Signal Processing on Graphs: Extending High-dimensional Data Analysis to Networks and Other Irregular Domains. *IEEE Signal Processing Magazine* 30, 3 (2013), 83–98.
- Martin Simonovsky and Nikos Komodakis. 2017. Dynamic Edge-Conditioned Filters in Convolutional Neural Networks on Graphs. In *Proc. CVPR*.
- Ayan Sinha, Jing Bai, and Karthik Ramani. 2016. Deep Learning 3D shape Surfaces using Geometry Images. In *Proc. ECCV*.
- Hang Su, Varun Jampani, Deqing Sun, Subhransu Maji, Evangelos Kalogerakis, Ming-Hsuan Yang, and Jan Kautz. 2018. SPLATNet: Sparse Lattice Networks for Point Cloud Processing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2530–2539.
- Hang Su, Subhransu Maji, Evangelos Kalogerakis, and Erik Learned-Miller. 2015. Multi-view Convolutional Neural Networks for 3D Shape Recognition. In *Proc. CVPR*.
- Jian Sun, Maks Ovsjanikov, and Leonidas Guibas. 2009. A Concise and Provably Informative Multi-scale Signature based on Heat Diffusion. *Computer Graphics Forum* 28, 5 (2009), 1383–1392.
- Maxim Tatarchenko, Alexey Dosovitskiy, and Thomas Brox. 2017. Octree Generating Networks: Efficient Convolutional Architectures for High-resolution 3D Outputs. In *Proc. ICCV*.
- Federico Tombari, Samuele Salti, and Luigi Di Stefano. 2011. A Combined Texture-shape Descriptor for Enhanced 3D Feature Matching. In *Proc. ICIP*.
- Oliver Van Kaick, Hao Zhang, Ghassan Hamarneh, and Daniel Cohen-Or. 2011. A Survey on Shape Correspondence. *Computer Graphics Forum* 30, 6 (2011), 1681–1707.
- Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2017. Graph Attention Networks. *arXiv:1710.10903* (2017).
- Shenlong Wang, Simon Suo, Wei-Chiu Ma, Andrei Pokrovsky, and Raquel Urtasun. 2018b. Deep Parametric Continuous Convolutional Neural Networks. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*.
- Xiaolong Wang, Ross Girshick, Abhinav Gupta, and Kaiming He. 2018a. Non-local Neural Networks. *CVPR* (2018).
- Lingyu Wei, Qixing Huang, Duygu Ceylan, Etienne Vouga, and Hao Li. 2016. Dense Human Body Correspondences using Convolutional Networks. In *Proc. CVPR*.
- Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 2015. 3D Shapenets: A Deep Representation for Volumetric Shapes. In *Proc. CVPR*.
- Cihang Xie, Yuxin Wu, Laurens van der Maaten, Alan Yuille, and Kaiming He. 2018. Feature Denoising for Improving Adversarial Robustness. *arXiv preprint arXiv:1812.03411* (2018).
- Yaoqing Yang, Chen Feng, Yiru Shen, and Dong Tian. 2018. FoldingNet: Point Cloud Auto-Encoder via Deep Grid Deformation. In *Proc. CVPR*.
- Li Yi, Vladimir G Kim, Duygu Ceylan, I Shen, Mengyan Yan, Hao Su, ARCEwu Lu, Qixing Huang, Alla Sheffer, Leonidas Guibas, et al. 2016. A Scalable Active Framework for Region Annotation in 3D Shape Collections. *TOG* 35, 6 (2016), 210.
- Yuke Zhu, Roozbeh Mottaghi, Eric Kolve, Joseph J. Lim, Abhinav Gupta, Li Fei-Fei, and Ali Farhadi. 2017. Target-driven Visual Navigation in Indoor Scenes using Deep Reinforcement learning. In *Proc. ICRA*.

Syed Afaq Ali Shah、Mohammed Bennamoun、Farid Boussaid 和 Amar A El-Sallam。2013 年。3D-Div：一种新型的局部表面描述符，用于特征匹配和成对范围图像配准。发表于《ICIP》会议论文集。Yiru Shen、Chen Feng、Yaoqing Yang 和 Dong Tian。2017 年。邻居有帮助：深度挖掘点云的局部结构。arXiv:1712.06760 (2017 年)。David I Shuman、Sunil K Narang、Pascal Frossard、Antonio Ortega 和 Pierre Vandergheynst。2013 年。《信号处理领域的新兴领域：将高维数据分析扩展到网络和其他不规则域》。《IEEE 信号处理杂志》第 30 卷第 3 期 (2013 年)，第 83-98 页。Martin Simonovsky 和 Nikos Komodakis。2017 年。卷积神经网络中图上的动态边缘条件滤波器。发表于《CVPR》会议论文集。Ayan Sinha、Jing Bai 和 Karthik Ramani。2016 年。使用几何图像深度学习三维形状表面。发表于《ECCV》会议论文集。Hang Su、Varun Jampani、Deqing Sun、Subhransu Maji、Evangelos Kalogerakis、MingHsuan Yang 和 Jan Kautz。2018 年。SPLATNet：用于点云的稀疏格网络。

云处理。在 IEEE 计算机视觉与模式识别会议论文集中。2530–2539。

Hang Su、Subhransu Maji、Evangelos Kalogerakis 和 Erik Learned-Miller。2015 年。《多视图卷积神经网络用于 3D 形状识别》。在 CVPR 会议论文集中。Jian Sun、Maks Ovsjanikov 和 Leonidas Guibas。2009 年。《基于热扩散的简洁且可证明信息丰富的多尺度签名》。Computer Graphics Forum 28, 5 (2009), 1383–1392。Maxim Tatarchenko、Alexey Dosovitskiy 和 Thomas Brox。2017 年。《八叉树生成网络：用于高分辨率 3D 输出的高效卷积架构》。

在 ICCV 会议论文集中。
Federico Tombari、Samuele Salti 和 Luigi Di Stefano。2011 年。《纹理-形状组合描述符用于增强 3D 特征匹配》。在 ICIP 会议论文集中。Oliver Van Kaick、Hao Zhang、Ghassan Hamarneh 和 Daniel Cohen-Or。2011 年。《形状对应关系综述》。Computer Graphics Forum 30, 6 (2011), 1681–1707。Petar Velickovic、Guillem Cucurull、Arantxa Casanova、Adriana Romero、Pietro Liò 和 Yoshua Bengio。2017 年。《图注意力网络》。arXiv:1710.10903 (2017)。
Shenlong Wang、Simon Suo、Wei-Chiu Ma、Andrei Pokrovsky 和 Raquel Urtasun。2018b。深度参数连续卷积神经网络。在 IEEE 计算机视觉与模式识别会议（CVPR）。

王晓龙，Ross Girshick，Abhinav Gupta，和何恺明。2018a。非局部神经网络。CVPR (2018)。魏凌宇，黄启兴，Duygu Ceylan，Etienne Vouga，和 Li Hao。2016。使用卷积网络的人体密集对应。在 CVPR 会议。吴志荣，宋舒然，Aditya Khosla，Yu Fisher，张立刚，唐晓鸥，和肖建雄。2015。3D Shapenets：体积形状的深度表示。在 CVPR 会议。
谢驰航，吴宇欣，Laurens van der Maaten，Alan Yuille，和何恺明。

2018。特征去噪以提高对抗鲁棒性。arXiv 预印本 arXiv:1812.03411 (2018)。

姚庆阳、陈峰、沈一儒和天田。2018 年。《FoldingNet：基于深度网格变形的点云自动编码器》。在 CVPR 会议论文集中。李毅、弗拉基米尔·G·金、杜伊古·切扬、沈一、闫梦岩、苏浩、ARCewu 卢、黄启兴、阿拉·谢弗、利奥尼达斯·吉巴斯等。2016 年。《三维形状集合中区域标注的可扩展主动框架》。TOG 35, 6 (2016), 210。朱玉，穆塔格希·鲁兹贝赫，埃里克·科夫，李约瑟夫·J·林，古普塔·阿比纳夫，李飞飞，阿里·法哈迪。2017 年。《使用深度强化学习的室内场景目标驱动视觉导航》。在 ICRA 会议论文集中。